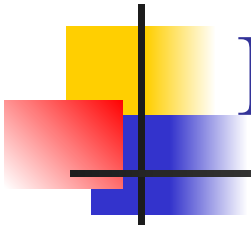


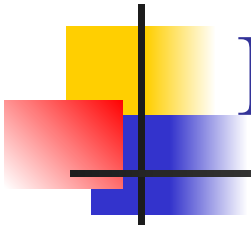
Chapter 8

Trees



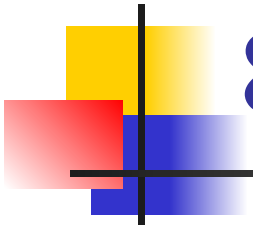
Lecture Outline (1/2)

- Why Trees
- What is a Tree?
- Rooted Trees
- Tree Terminology
- Ordered Trees/N-ary Trees
- Binary Tree
  - Representation of Binary Trees



Lecture Outline (2/2)

- Traversal in Binary Trees
- Types of Binary Trees
- Arithmetic Expression Trees
  - Extended Binary Trees
  - Weighted Binary Trees
- Huffman Trees
- Threaded Binary Trees
- General Trees
- Forest
  - Forest Traversals
- Application of Binary Trees



8.1 Why Trees

- Linked Lists provide greater flexibility than arrays but linear access time of linked lists is prohibitive
- Stacks and queues reflect some hierarchy but they are limited to one dimension
- To overcome these limitation we create trees
  - They are simple data structure for which the running time of most operations (search, insert, delete) is O(log N)?

Example: UNIX Directory

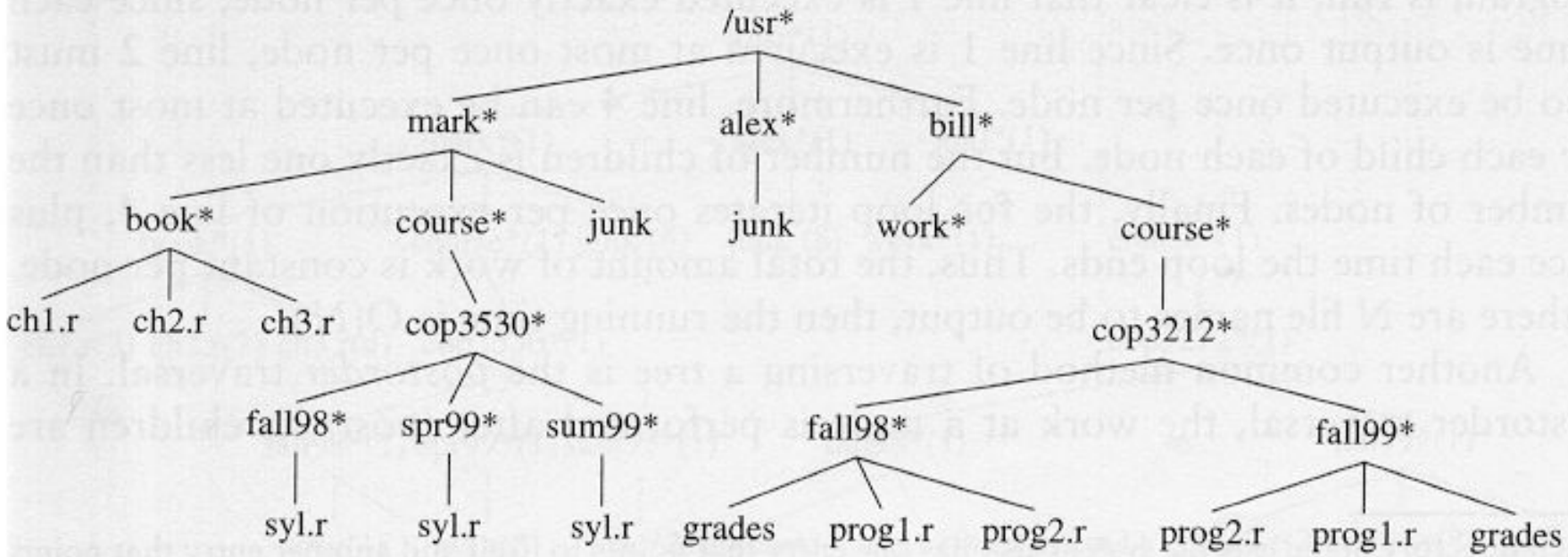
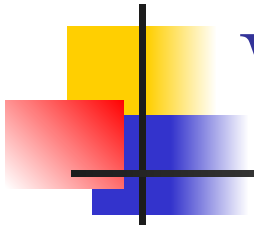


Figure 4.5 UNIX directory



Why Trees?

- Trees are very important data structures in computing.
- They are suitable for:
  - Hierarchical structure representation, e.g.,
    - File directory.
    - Organizational structure of an institution.
    - Class inheritance tree.
  - Problem representation, e.g.,
    - Expression tree.
    - Game tree.
    - Decision tree.
  - Efficient algorithmic solutions, e.g.,
    - Search trees.
    - Efficient priority queues via heaps.

DS1,P-262

Trees

- A tree is a collection of nodes
  - The collection can be empty
  - (recursive definition) If not empty, a tree consists of a distinguished node r (the *root*), and zero or more nonempty *subtrees* T_1, T_2, ..., T_k, each of whose roots are connected by a directed *edge* from r

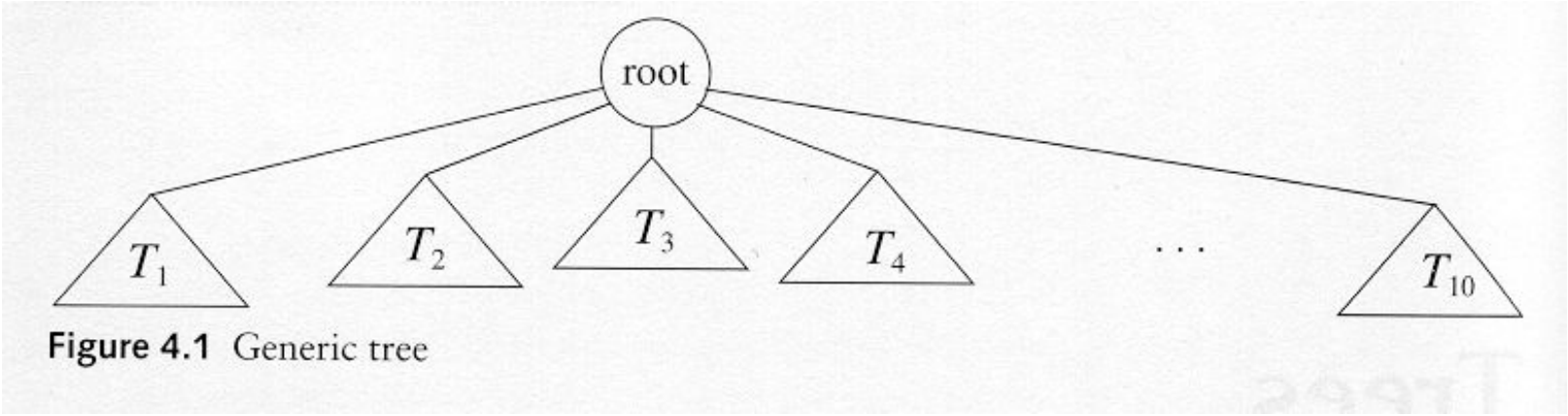


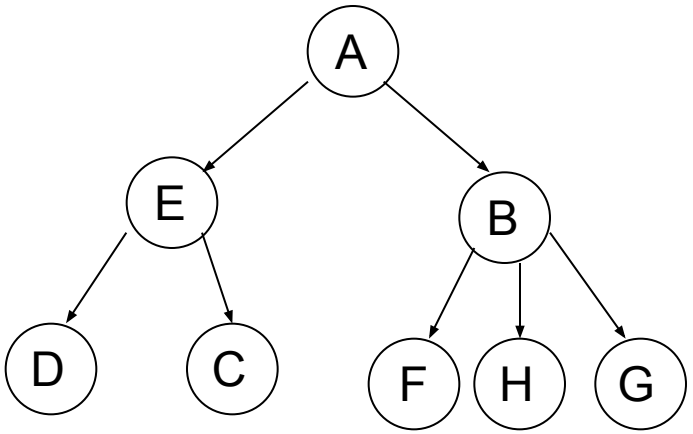
Figure 4.1 Generic tree

What is a Tree? (1/2)

- A tree, is a finite set of nodes together with a finite set of directed edges that define parent-child relationships. Each directed edge connects a parent to its child. Example:

Nodes={A,B,C,D,E,f,G,H}

Edges={(A,B),(A,E),(B,F),(B,G),(B,H),
(E,C),(E,D)}

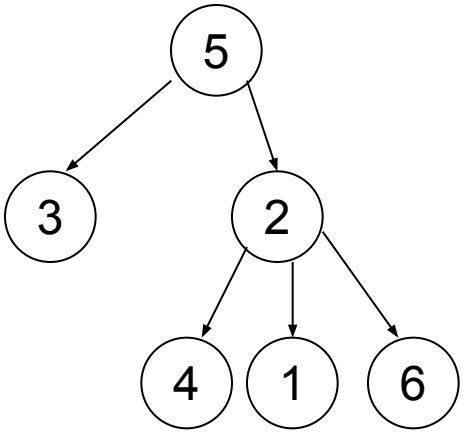


- A directed path from node m1 to node m_k is a list of nodes m_1, m_2, . . . , m_k such that each is the parent of the next node in the list. The length of such a path is k - 1.
- Example: A, E, C is a directed path of length 2.

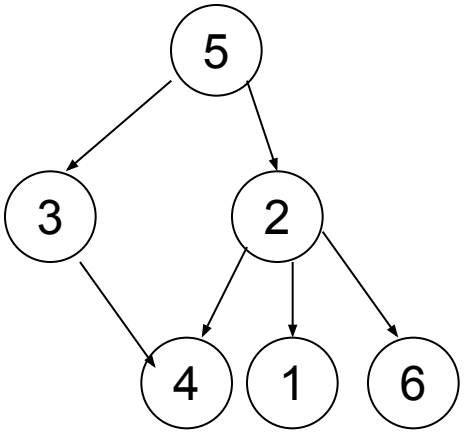
What is a Tree? (2/2)

- A tree satisfies the following properties:

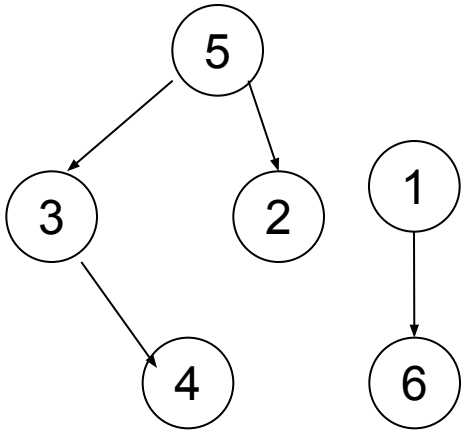
1. It has one designated node, called the root, that has no parent.
2. Every node, except the root, has exactly one parent.
3. A node may have zero or more children.
4. There is a unique directed path from the root to each node.



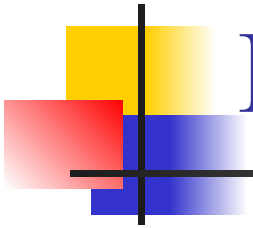
tree



Not a tree



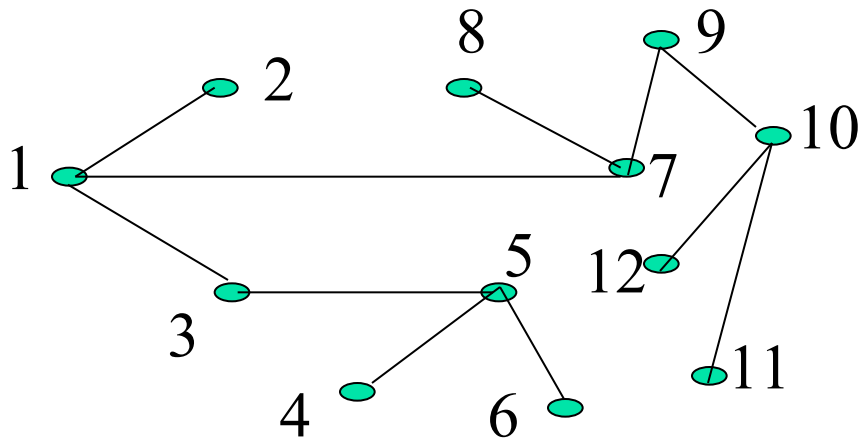
Not a tree



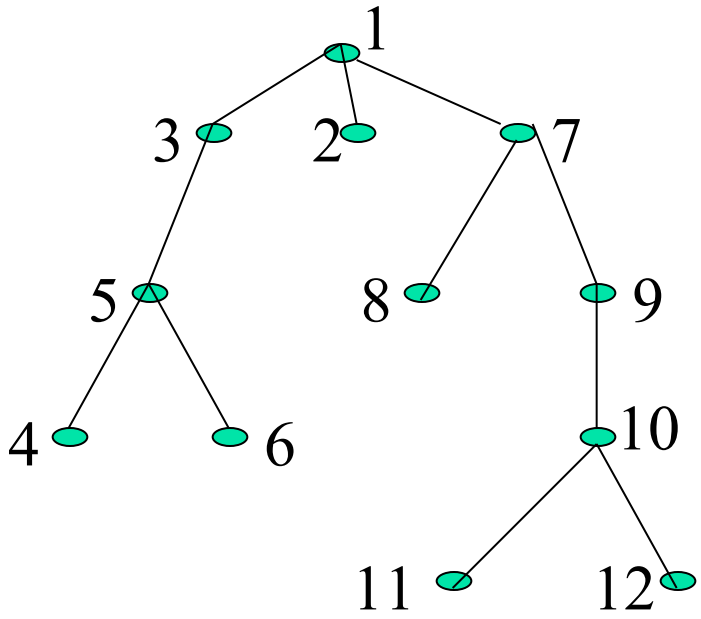
Rooted Trees

- A rooted tree is a tree where one of the nodes is designated as the root node. (Only one root in a tree)
- A rooted tree has a hierarchical structure: the root on top, followed by the nodes adjacent to it right below, followed by the nodes adjacent to those next, and so on.

Example of a Rooted Tree



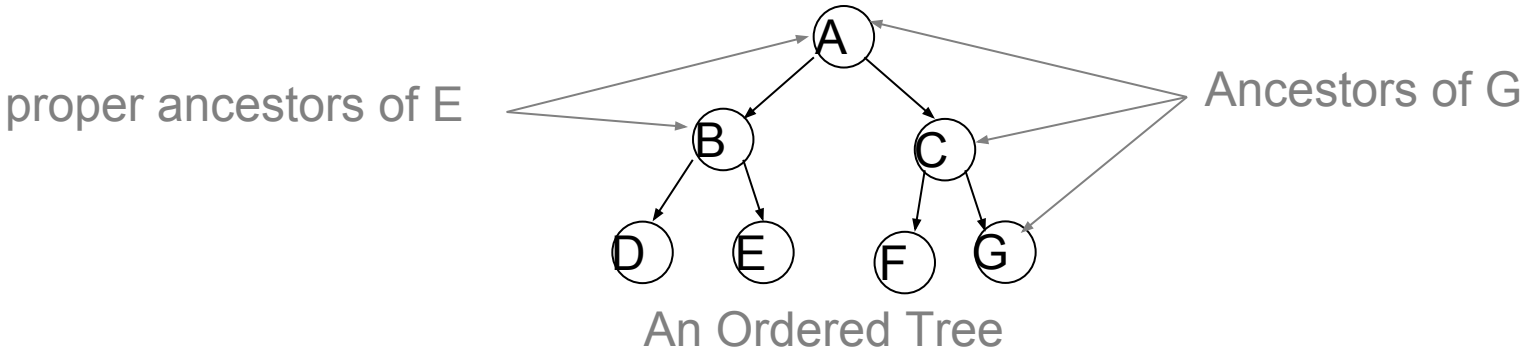
Unrooted tree



Tree rooted with root 1

8.2 Tree Terminology (1/5)

- **Ordered tree**: A tree in which the children of each node are linearly ordered (usually from left to right).

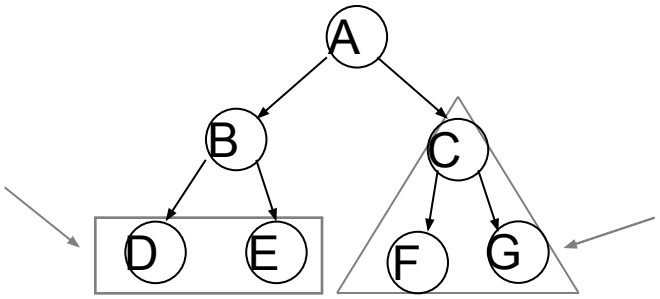


- **Ancestor of a node v**: Any node, including v itself, on the path from the root to the node.
- **Proper ancestor of a node v**: Any node, excluding v, on the path from the root to the node.

Tree Terminology (2/5)

- **Descendant of a node v**: Any node, including v itself, on any path from the node to a leaf node (i.e., a node with no children).

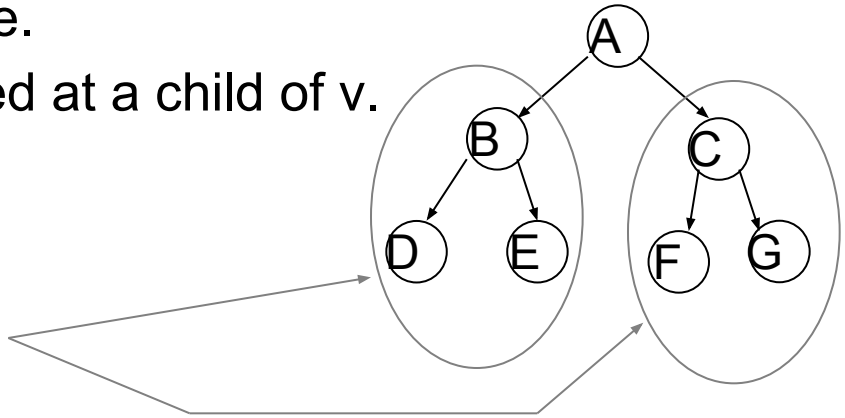
Proper descendants of node B



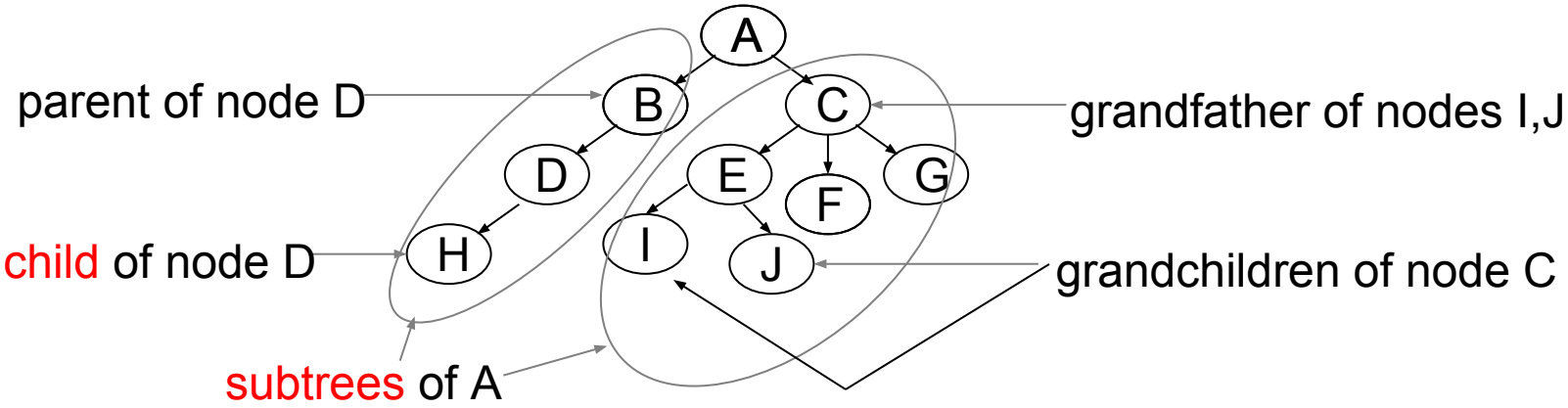
Descendants of a node C

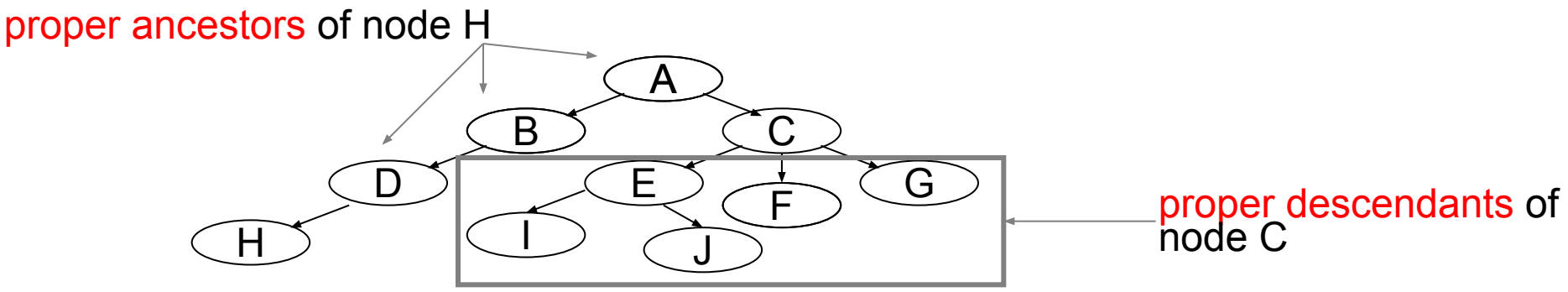
- **Proper descendant of a node v**: Any node, excluding v, on any path from the node to a leaf node.
- Subtree of a node v: A tree rooted at a child of v.

subtrees of node A



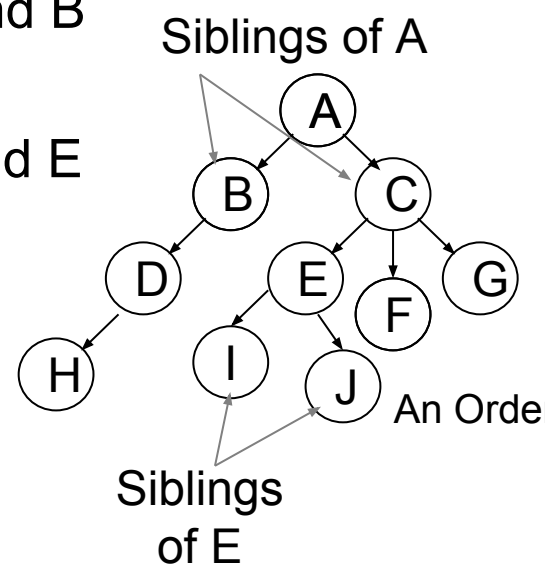
Tree Terminology (3/5)





Tree Terminology (4/5)

- Each of node D and B has degree 1.
- Each of node A and E has degree 2.



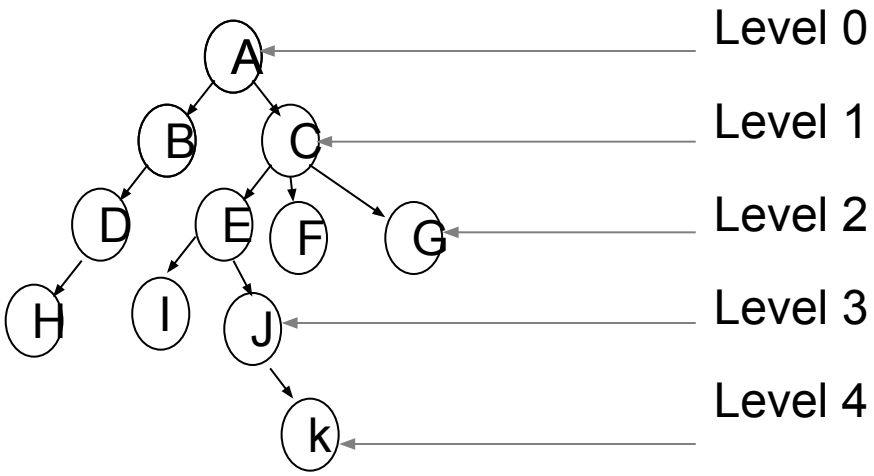
An Ordered Tree with size of 10

- node C has degree 3.
- Each of node F,G,H,I,J has degree 0.

- Leaf: A node with degree 0.
- Internal or interior node: a node with degree greater than 0.
- Siblings: Nodes that have the same parent.
- Size: The number of nodes in a tree.

Tree Terminology (5/5)

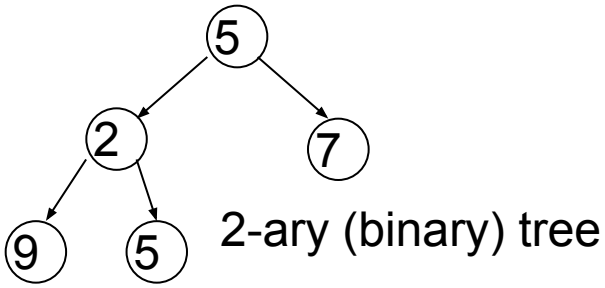
- The height of the tree is 4.
- The height of node C is 3.



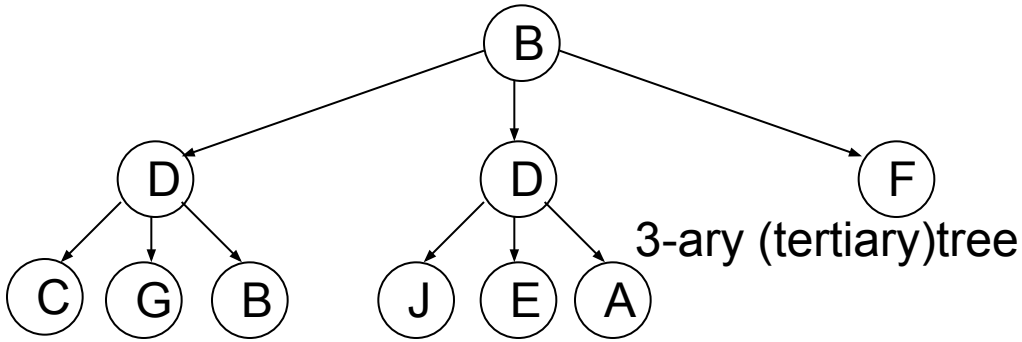
- Level (or depth) of a node v: The length of the path from the root to v.
- Height of a node v: The length of the longest path from v to a leaf node.
  - Height of a node v: The length of the longest path from v to a leaf node.
  - The height of a tree is the height of its root mode.
  - By definition the height of an empty tree is -1.

Ordered Trees/
N-ary Trees

- An N-ary tree is an ordered tree that is either:
  1. Empty, or
  2. It consists of a root node and at most N non-empty N-ary subtrees.
- It follows that the degree of each node in an N-ary tree is at most N.
- Example of N-ary trees:



2-ary (binary) tree



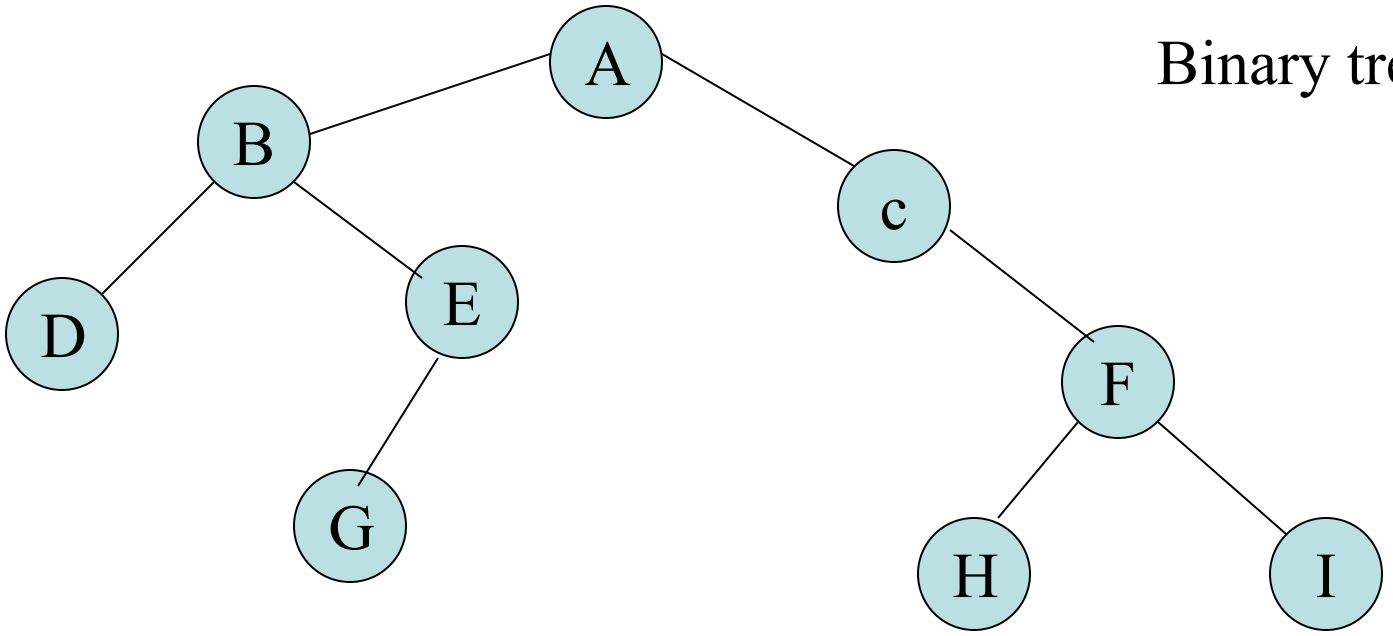
3-ary (tertiary)tree

Binary Tree

A binary tree is a finite set of elements that is either empty or is partitioned into three disjoint subsets.

- The first subset contains a single element called the root of the tree.
- The other two subsets are themselves binary trees, called the left and right subtrees of the original tree. A left or right subtree can be empty.

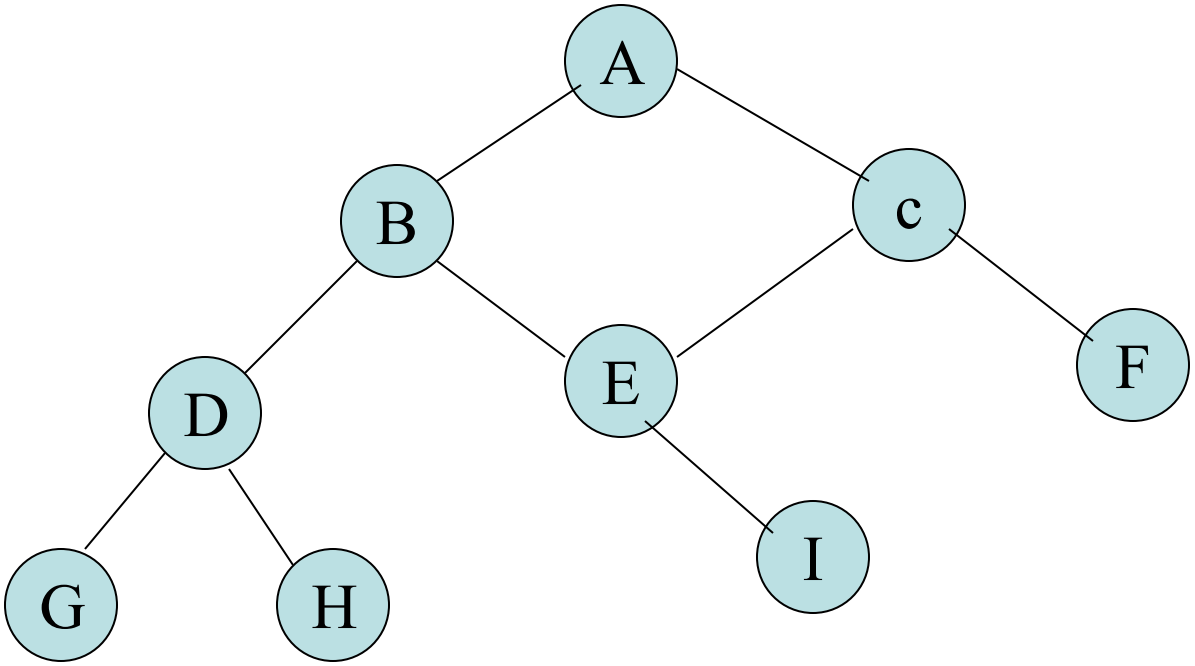
Each element of a tree is called a node of the tree.



Binary tree

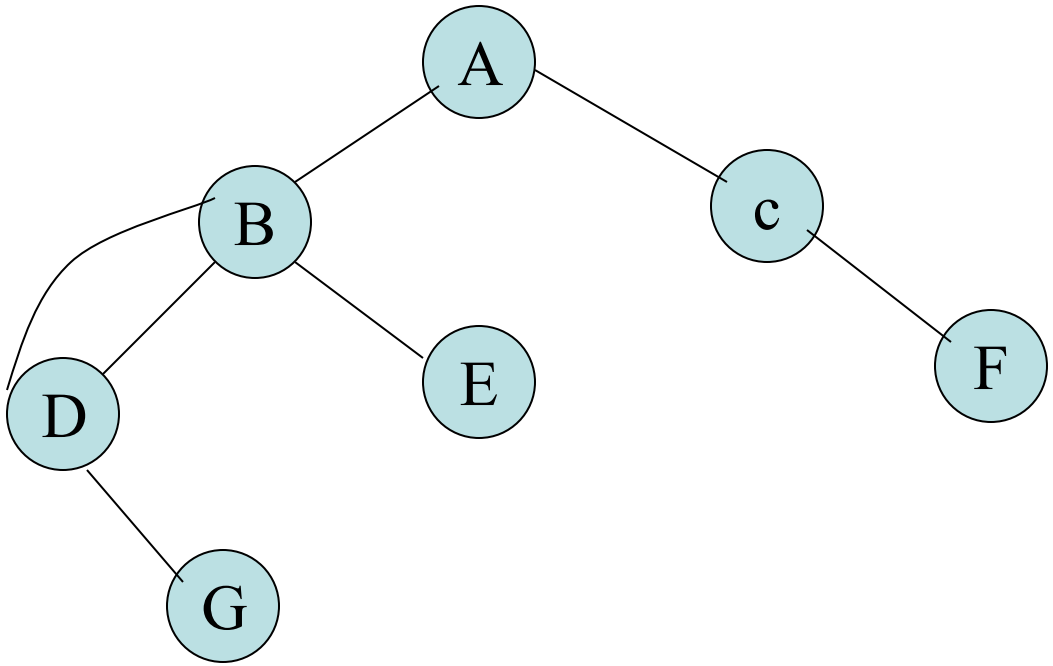
Binary Tree

Structures that are not binary trees



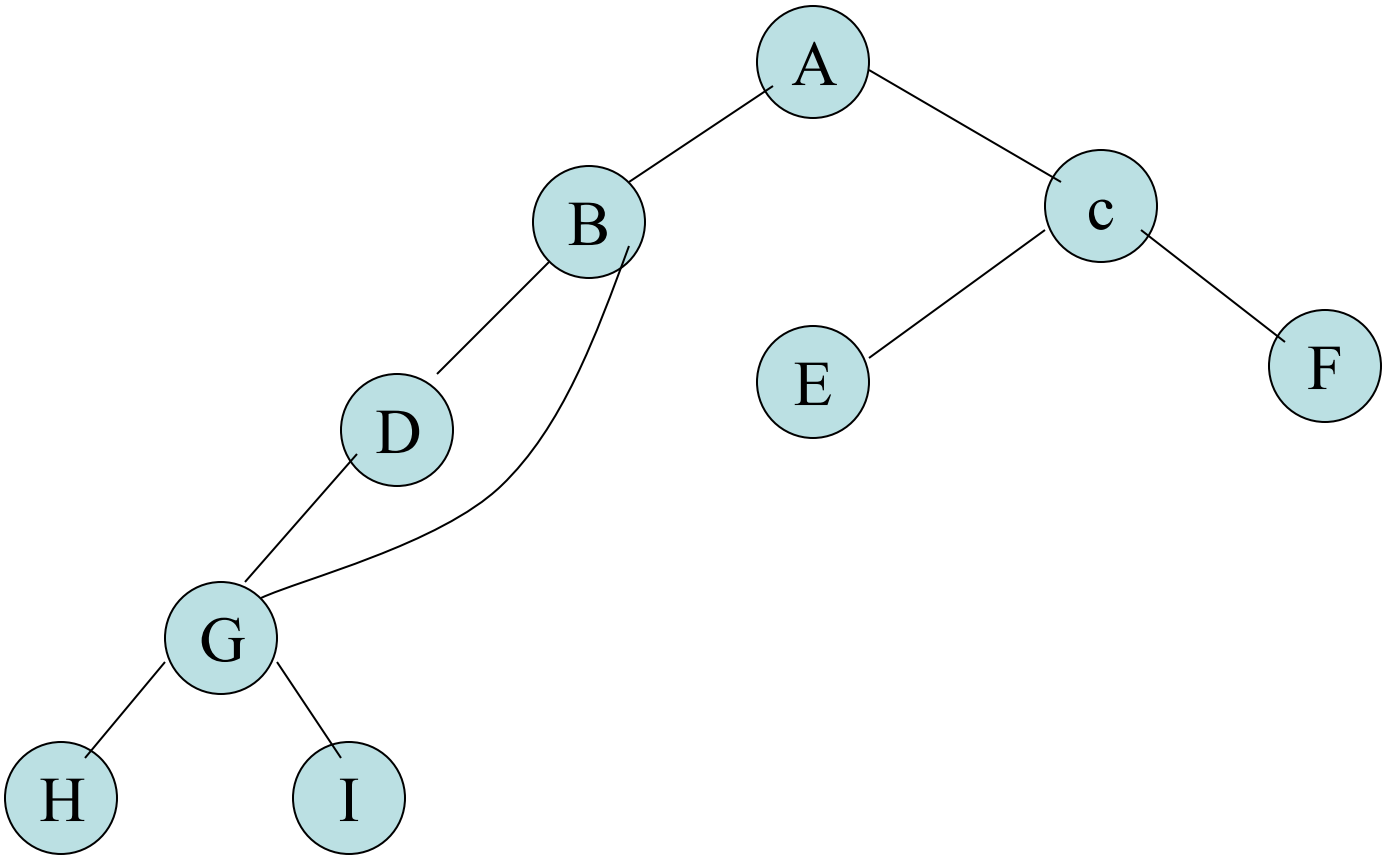
Binary Tree

Structures that are not binary trees



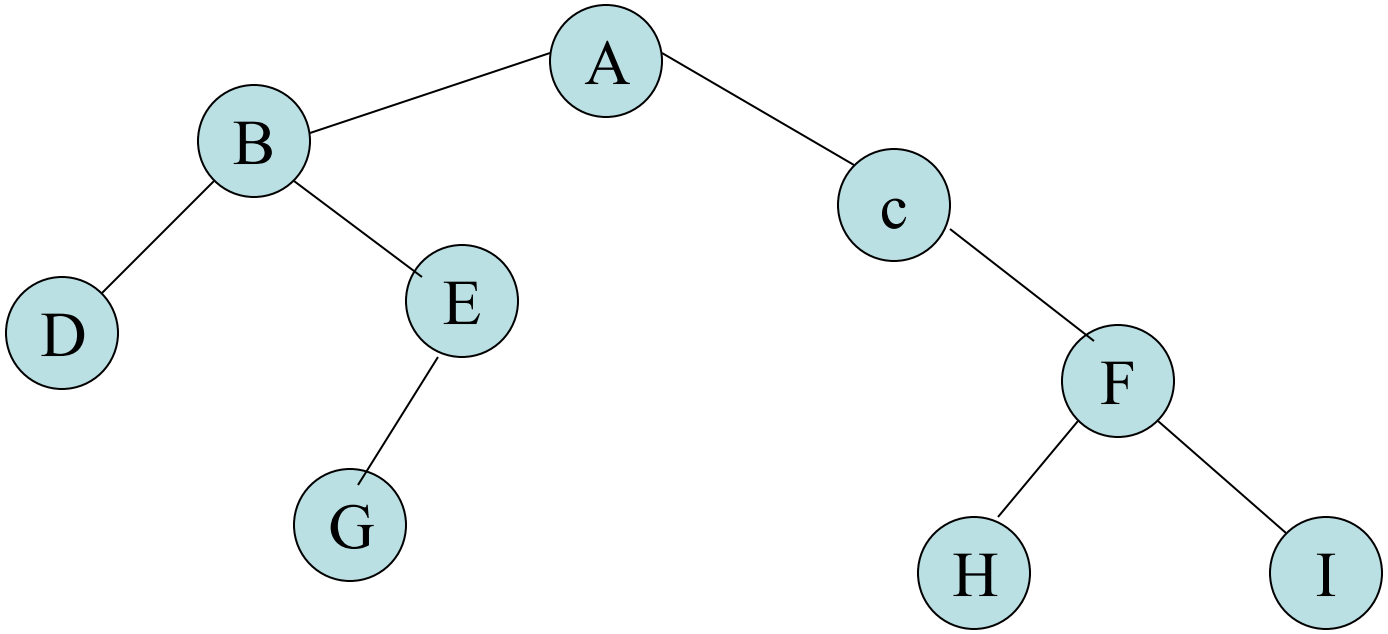
Binary Tree

Structures that are not binary trees

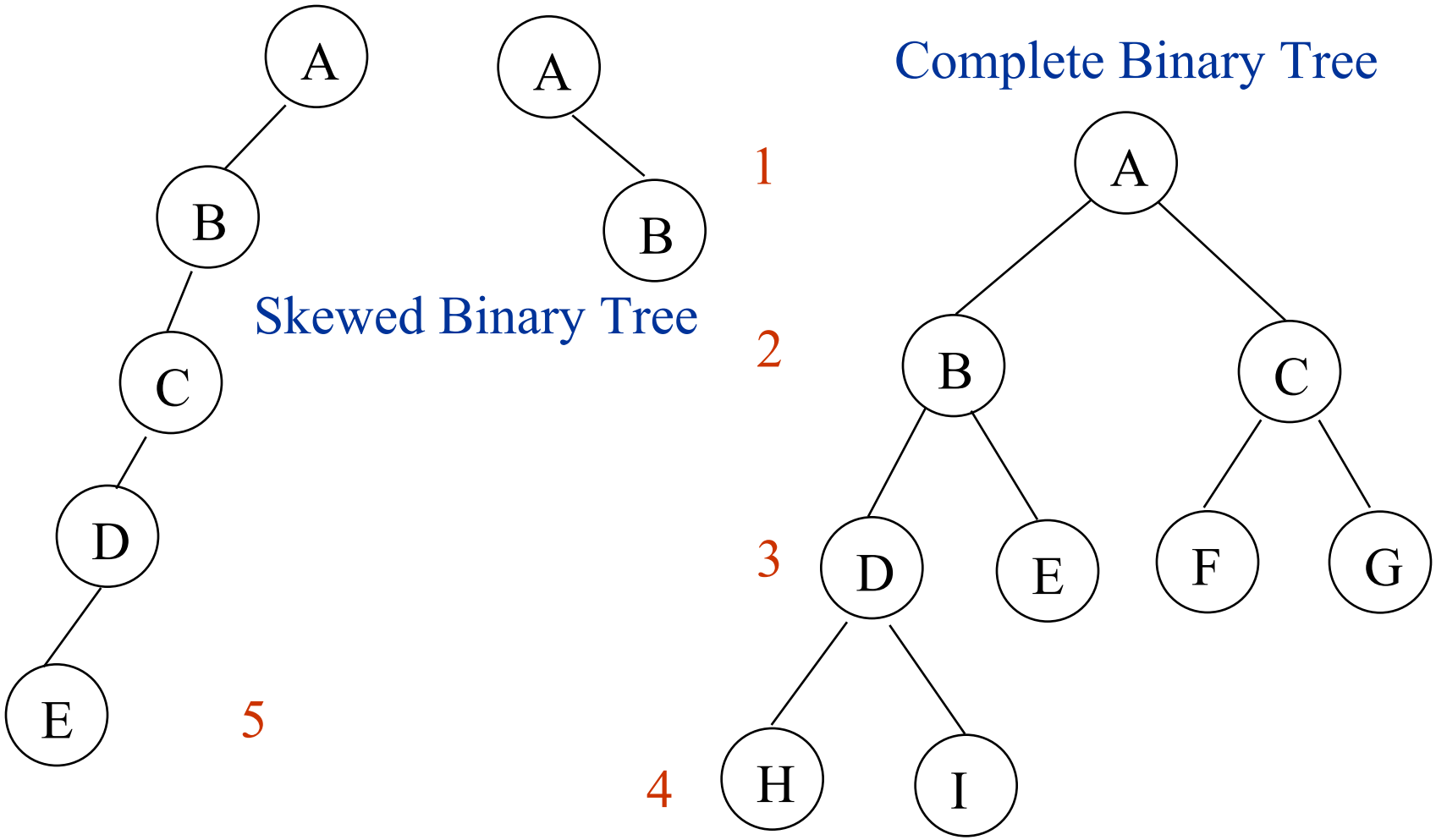


Binary tree

Structure that is not a strictly binary tree:
because nodes C and E have one son each.



Binary tree


A
B
C
D
E
5
A
B
Skewed Binary Tree
1
2
3
4
Complete Binary Tree
A
B
C
D
E
F
G
H
I

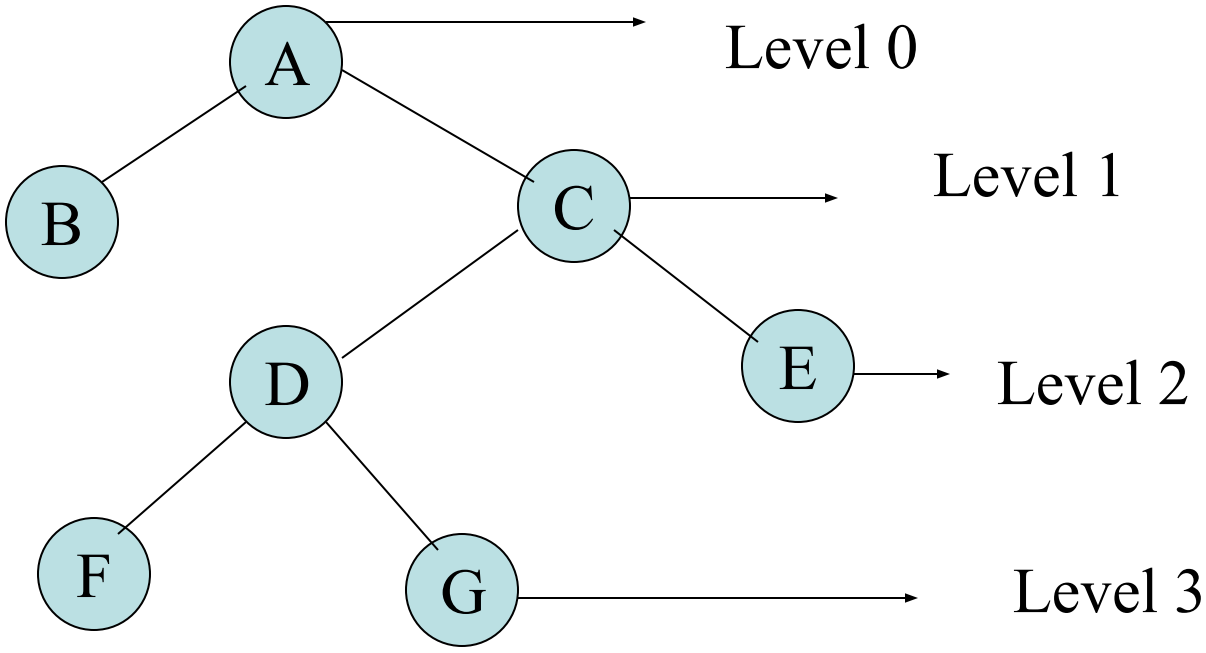
Level and depth of a binary Tree

Level of binary tree:

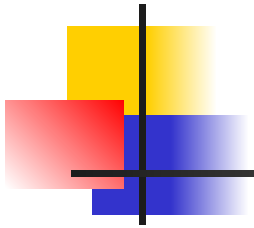
The root of the tree has level 0. And the level of any other node is one more than the level of its father.

Depth of a binary tree:

The depth of a binary tree is the maximum level of any leaf in the tree.



Depth is 3.

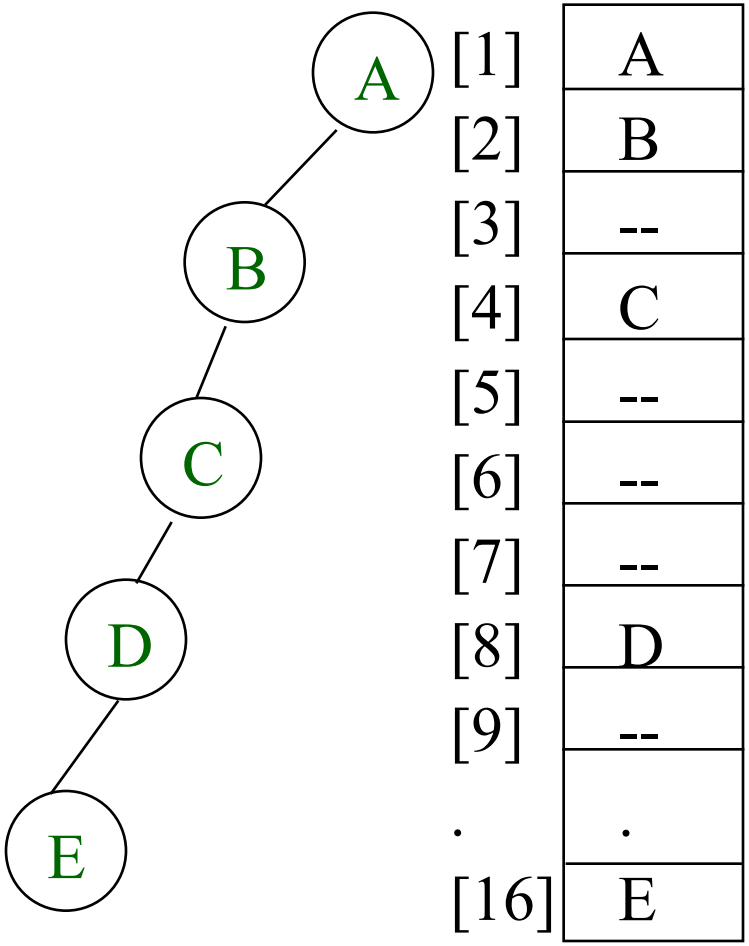


8.4 Representation of Binary Trees

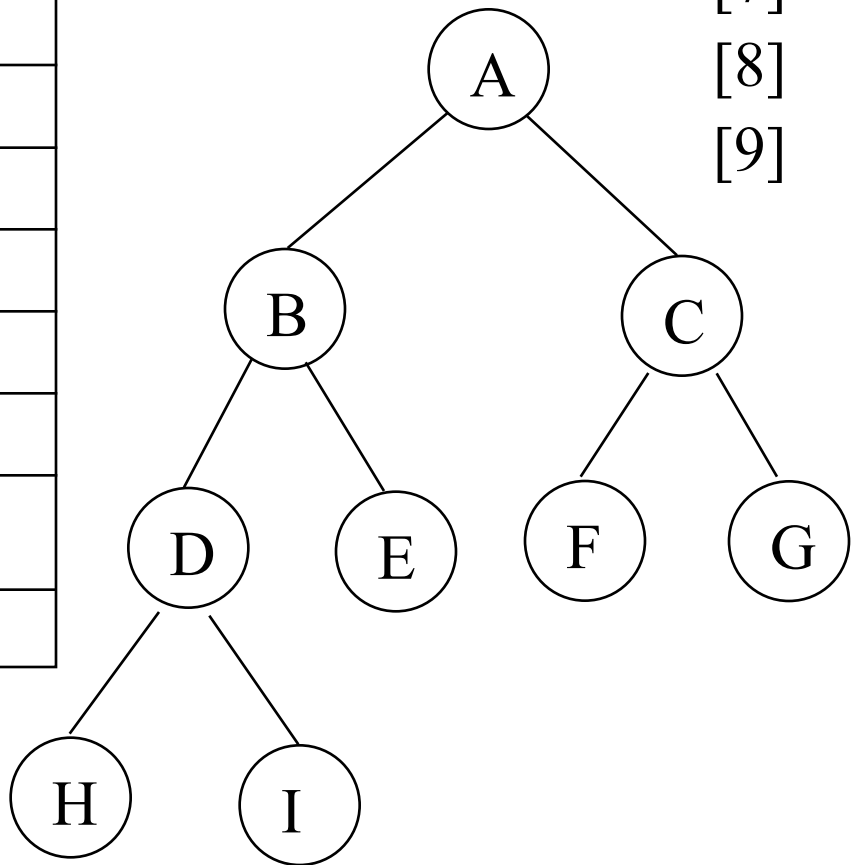
Binary Tree Representations

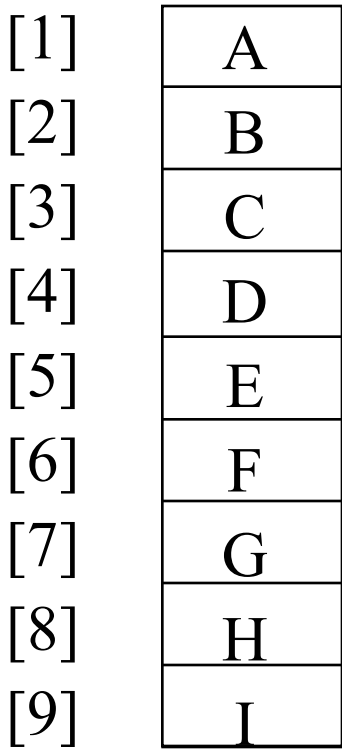
- If a complete binary tree with *n* nodes (depth = log *n* + 1) is represented sequentially, then for any node with index *i*, 1<=*i*<=*n*, we have:
  - *parent*(*i*) is at *i*/2 if *i*!=1. If *i*=1, *i* is at the root and has no parent.
  - *left_child*(*i*) ia at 2*i* if 2*i*<=*n*. If 2*i*>n, then *i* has no left child.
  - *right_child*(*i*) ia at 2*i*+1 if 2*i* +1 <=*n*. If 2*i* +1 >n, then *i* has no right child.

Sequential Representation

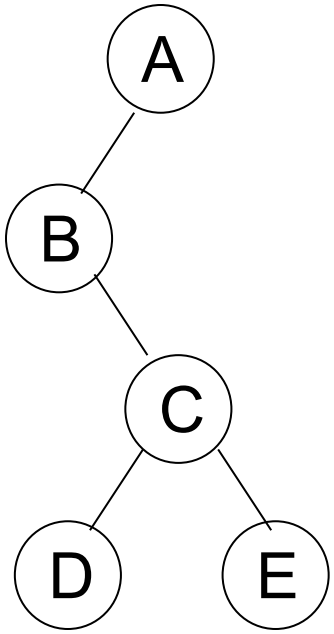


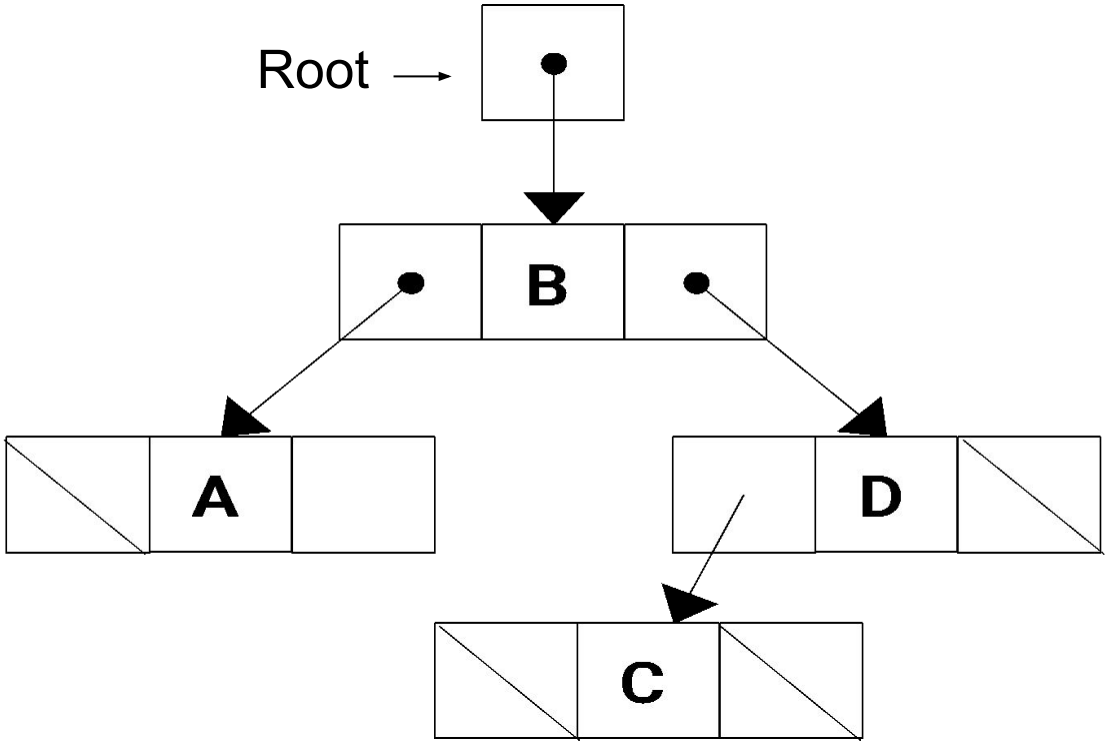
(1) waste space
(2) insertion/deletion problem



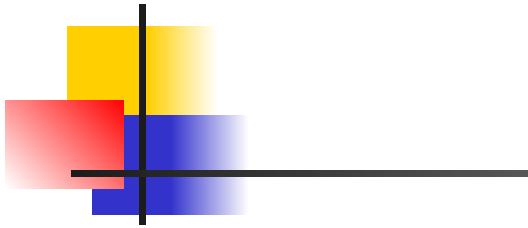


Linked List Representation





Tree structure using Doubly Linked List

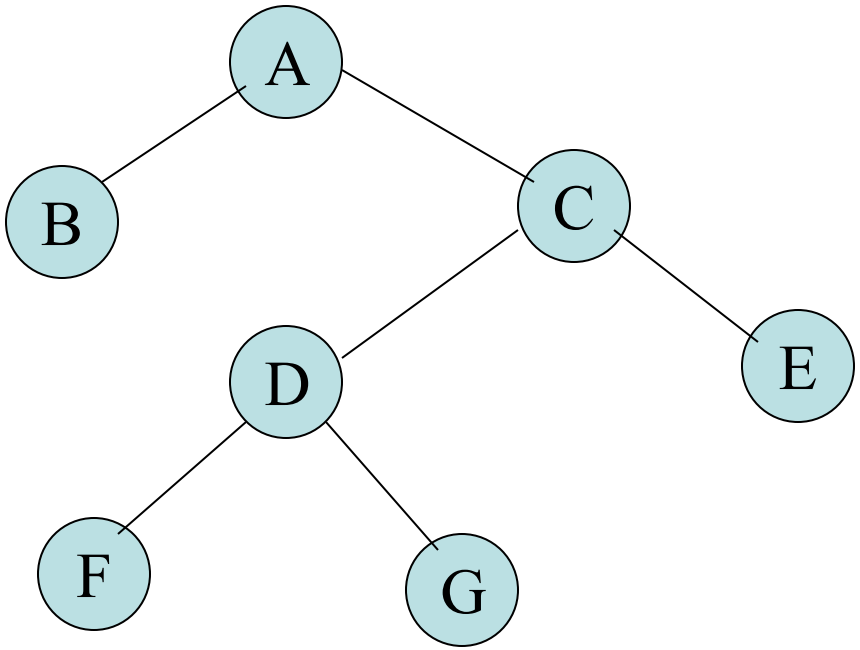


8.5 Types of Binary Trees

Binary Tree

8.6 Strictly binary trees:

If every nonleaf node in a binary tree has nonempty left and right subtrees, the tree is called a strictly binary tree.
A strictly binary tree with n leaves always contains 2n -1 nodes.



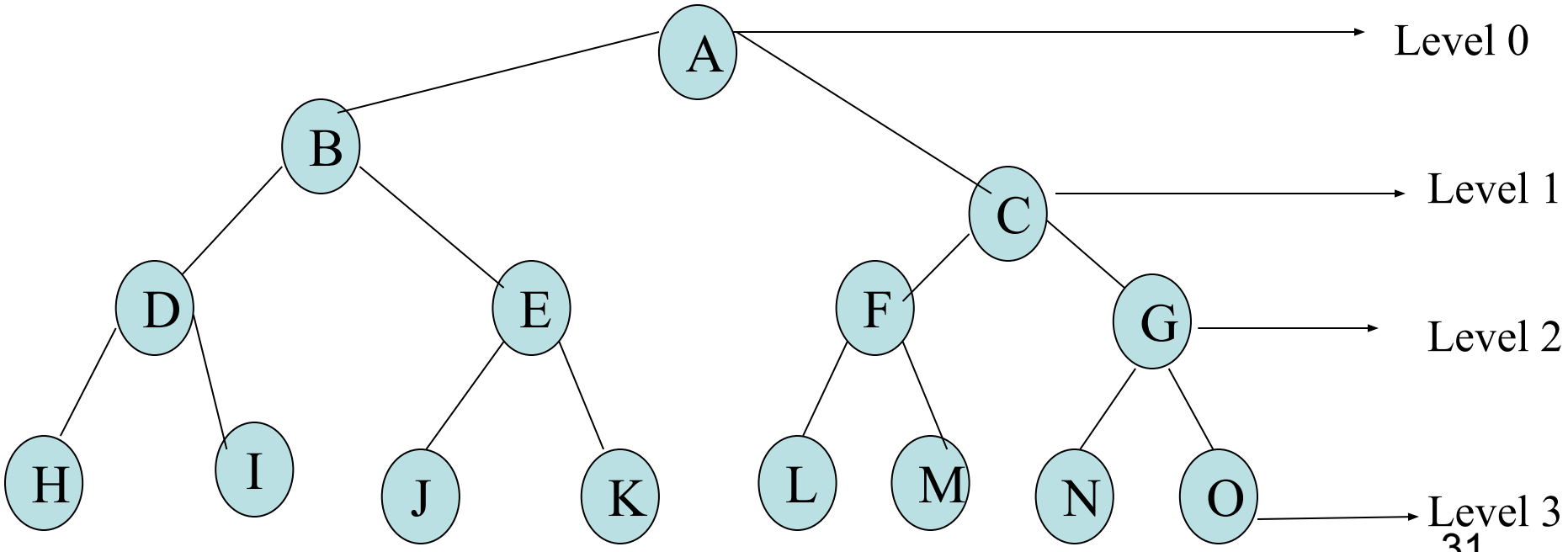
8.7 A complete binary tree

Complete binary tree of depth d is the strictly binary tree all of whose leaves are at level d.

A complete binary tree of depth d is the binary tree of depth d that contains exactly 2^{l} nodes at each level between 0 and d.

The total number of nodes = the sum of the number of nodes at each level between 0 and d.

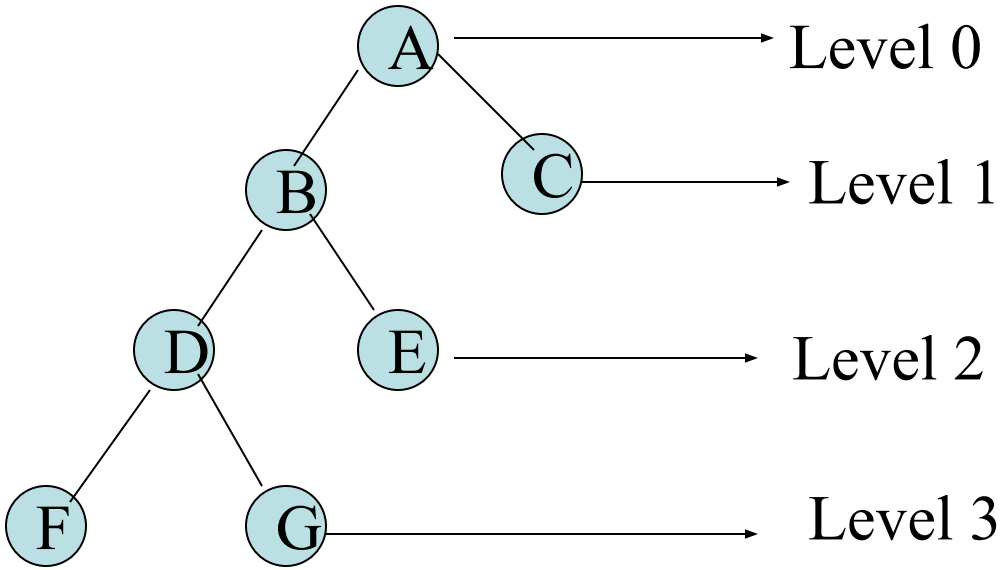
$= 2^{d+1} - 1$



8.8 Almost complete binary tree (1/4)

A binary tree of depth d is an almost complete binary tree if:

- 1. A node at level less than d -1 has two sons
- 2. For any node in the tree with a right descendant at level d, node must have a left son and every left descendant of node is either a leaf at level d or has two sons.



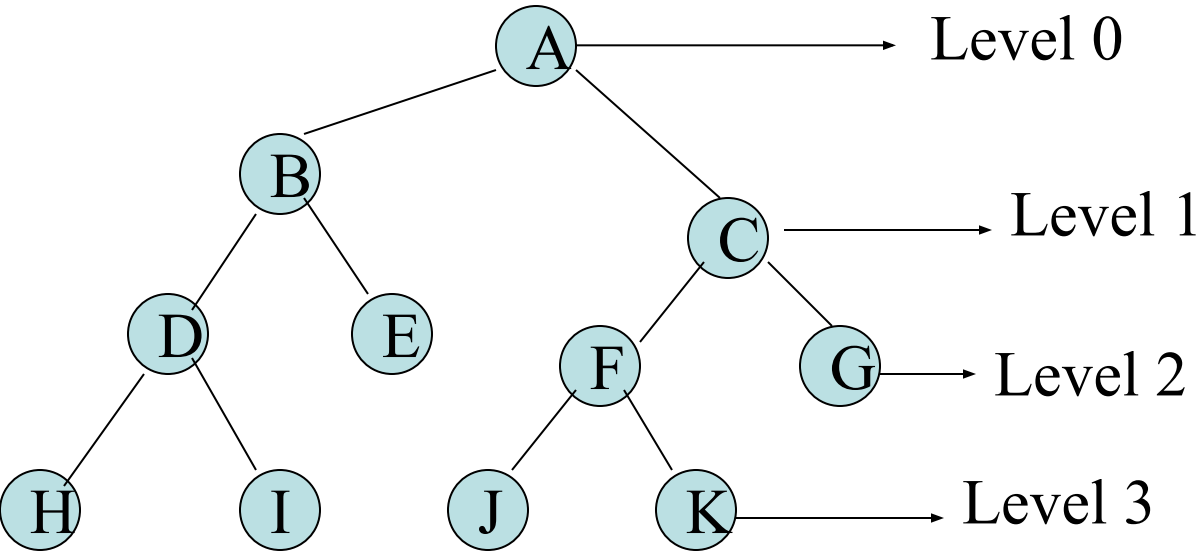
The strictly binary tree is not almost complete, since it contains leaves at levels 1, 2, and 3.

Violates condition 1

Almost complete binary tree (2/4)

A binary tree of depth d is an almost complete binary tree if:

- 1. A node at level less than d -1 has two sons
- 2. For any node in the tree with a right descendant at level d, node must have a left son and every left descendant of node is either a leaf at level d or has two sons.



d-1=2

The strictly binary tree is not almost complete, since A has a right descendant at level 3 (J) but also has a left descendant that is a leaf at level 2 (E)

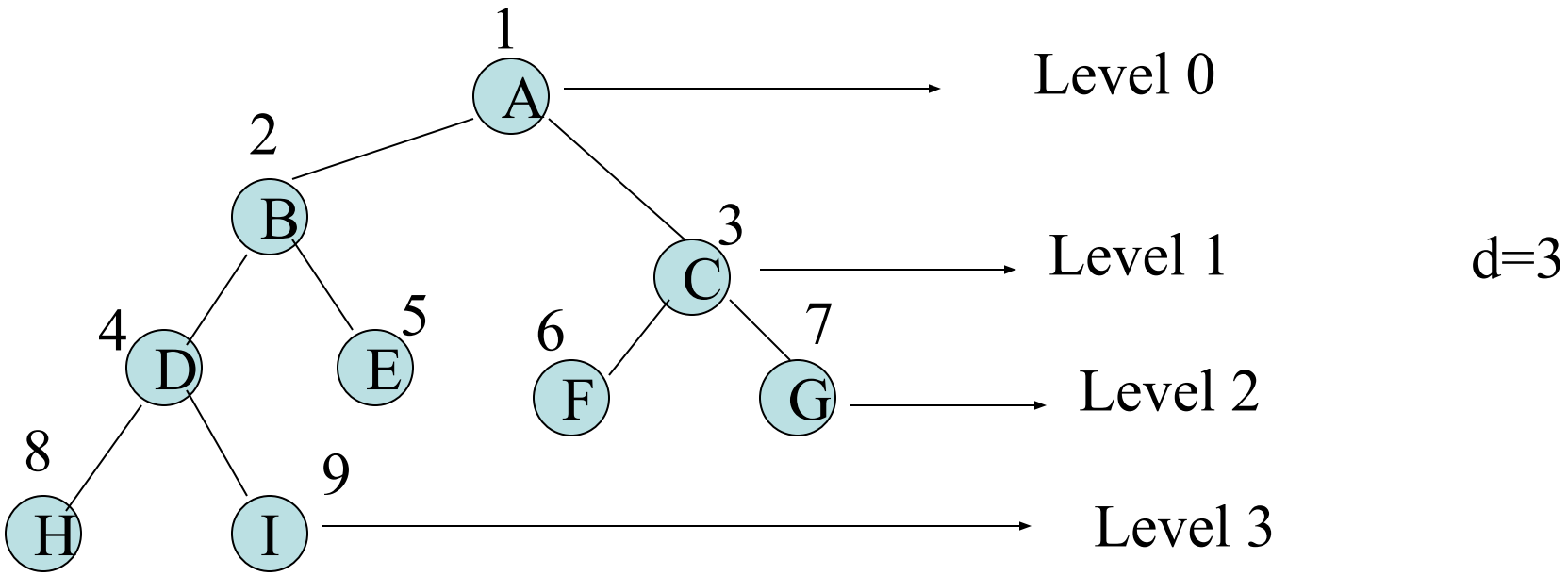
Violates condition 2

Satisfies the condition 1, since every leaf node is either at level 2 or at level 3.

Almost complete binary tree (3/4)

A binary tree of depth d is an almost complete binary tree if:

- 1. A node at level less than d -1 has two sons
- 2. For any node in the tree with a right descendant at level d, node must have a left son and every left descendant of node is either a leaf at level d or has two sons.

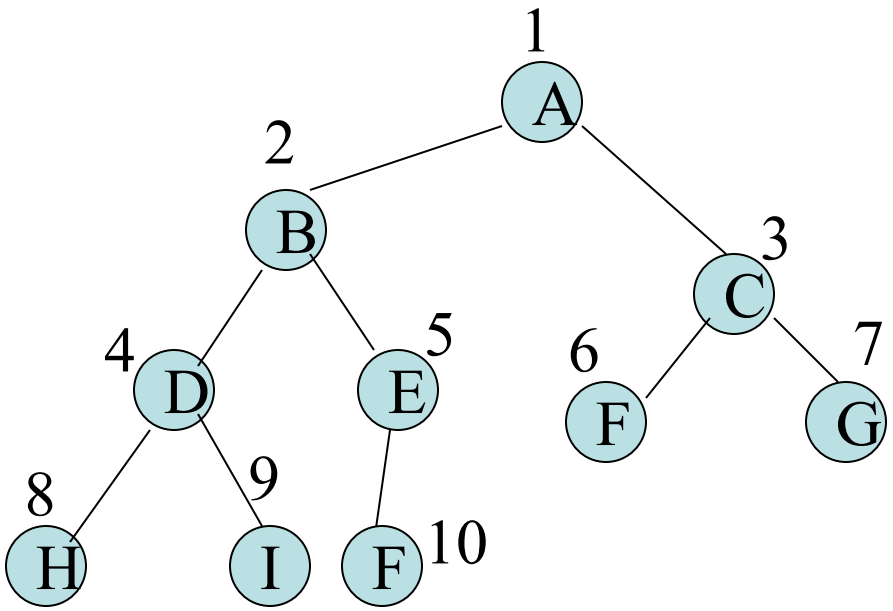


The binary tree is almost complete, Satisfies the condition1, since every leaf node is either at level 2 or at level 3.
Satisfies the condition 2

Almost complete binary tree (4/4)

A binary tree of depth d is an almost complete binary tree if:

- 1. A node at level less than d -1 has two sons
- 2. For any node in the tree with a right descendant at level d, node must have a left son and every left descendant of node is either a leaf at level d or has two sons.



The binary tree
is almost complete,
Satisfies the condition1, since every leaf node is either at level 2 or at level 3.

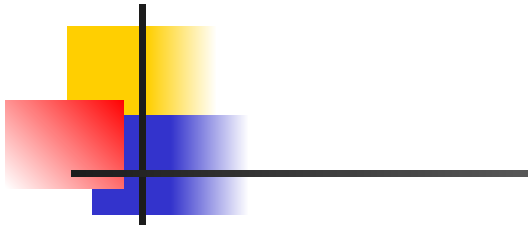
Satisfies the condition 2

However, the binary tree is not strictly binary tree, since node E has a left son but not a right son

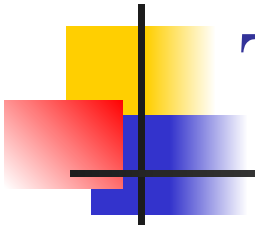
Bound on number of nodes in Almost complete binary tree

- An almost complete binary tree of depth d is intermediate between the complete binary tree of depth d-1 that contains 2^d-1 nodes and complete binary tree of depth d , which contains 2^{d+1}-1 nodes

DS1,P-275



8.9 Traversal in Binary Trees



Tree Traversal

- The process of systematically visiting all the nodes in a tree and performing some computation at each node in the tree is called a tree traversal.
- There are two methods in which to traverse a tree:
  1. Depth-First Traversal.
  2. Breadth-First Traversal.

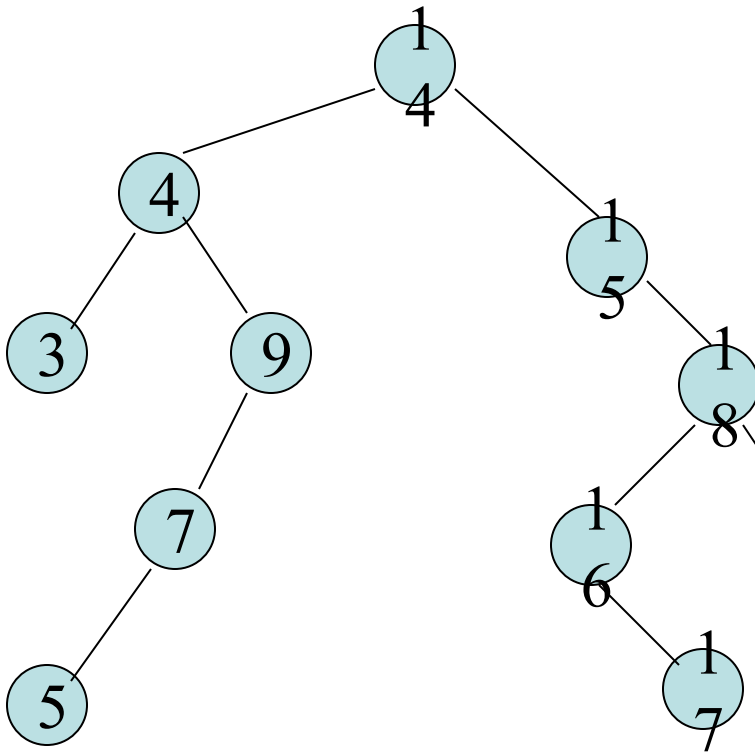
Depth-First Traversal

Name	Each Node
Preorder (N-L-R)	▪ Visit the node ▪ Visit the left subtree, if any. ▪ Visit the right subtree, if any.
Inorder (L-N-R)	▪ Visit the left subtree, if any. Visit the node ▪ Visit the right subtree, if any.
Postorder (L-R-N)	▪ Visit the left subtree, if any. ▪ Visit the right subtree, if any. ▪ Visit the node

8.9 Traversing a binary tree

Three methods:

- 1. preorder
- 2. inorder
- 3 postorder



Preorder

1. Visit the root
2. Traverse the left subtree in preorder
3. Traverse the right subtree in preorder

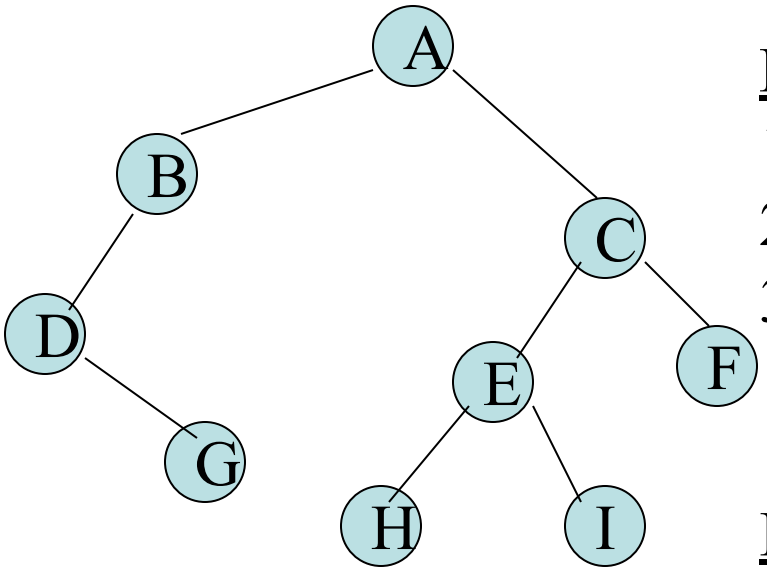
Inorder

1. Traverse the left subtree in inorder
2. Visit the root
3. Traverse the right subtree in inorder

Postorder

1. Traverse the left subtree in postorder
2. Traverse the right subtree in postorder
3. Visit the root

Traversing a binary tree



Preorder

1. Visit the root
2. Traverse the left subtree in preorder
3. Traverse the right subtree in preorder

preorder: ABDGCEHIF

Inorder

1. Traverse the left subtree in inorder
2. Visit the root
3. Traverse the right subtree in inorder

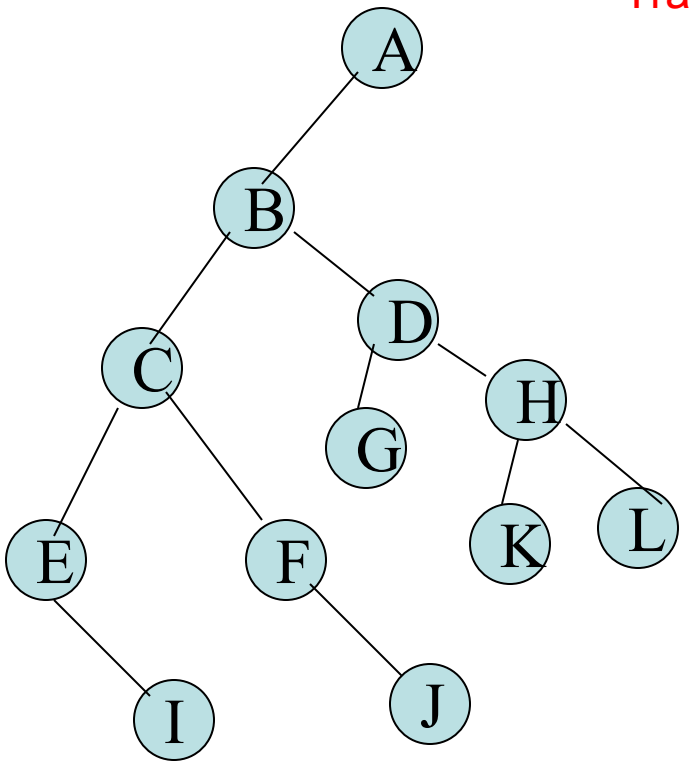
inorder: DGBAHEICF

Postorder

1. Traverse the left subtree in postorder
2. Traverse the right subtree in postorder
3. Visit the root

postorder: GDBHIEFCA

Traversing a binary tree



Preorder

1. Visit the root
2. Traverse the left subtree in preorder
3. Traverse the right subtree in preorder

preorder: ABCEIF JDGHKL

Inorder

1. Traverse the left subtree in inorder
2. Visit the root
3. Traverse the right subtree in inorder

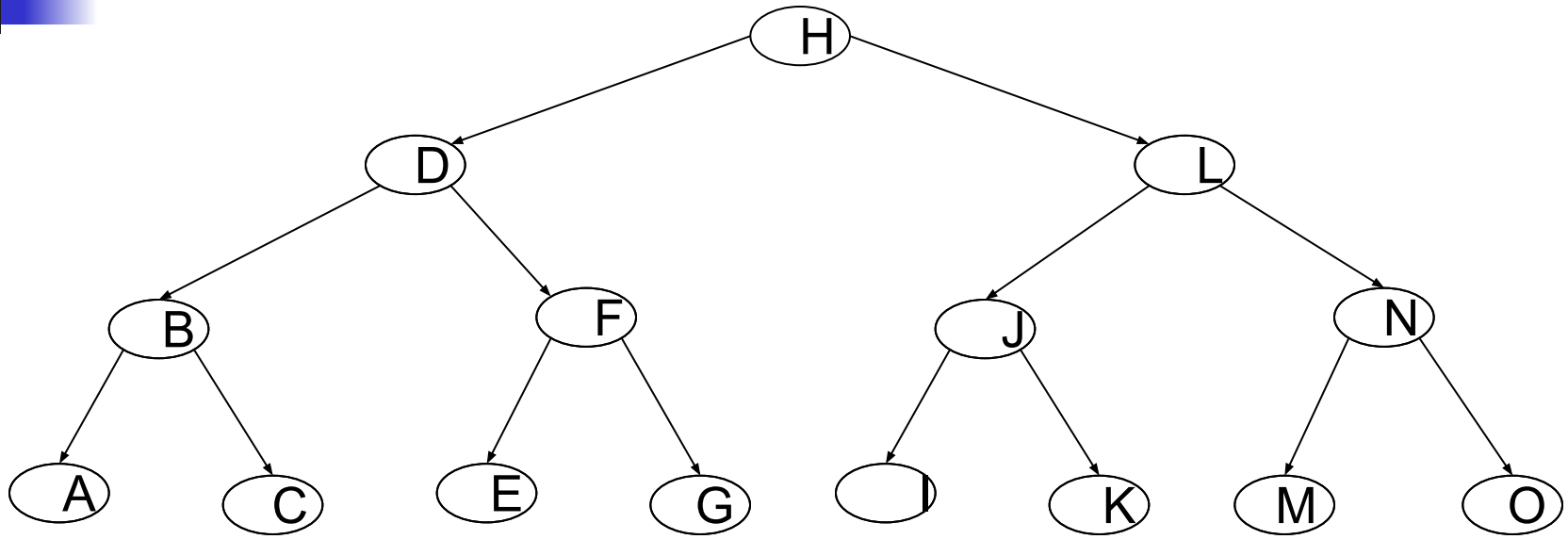
inorder: EICFJB GDKHLA

Postorder

1. Traverse the left subtree in postorder
2. Traverse the right subtree in postorder
3. Visit the root

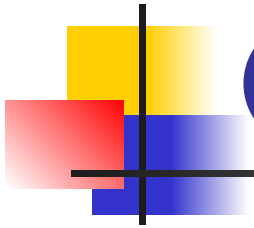
postorder: IEJFCG KLHDBA

Breadth-First Traversal



H	D	L	B	F	J	N	A	C	E	G	I	K	M	O

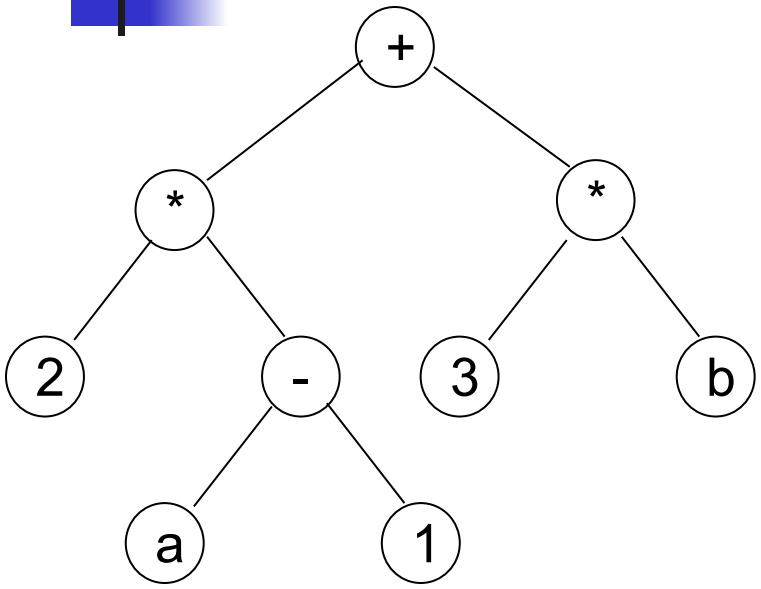
8.12 Arithmetic Expression Trees (1/2)



- Binary Tree associated with Arithmetic expression is called Arithmetic Expression Tree
- All the internal nodes are operators
- All the external nodes are operands
- When Arithmetic Expression tree is traversed in in-order, the output is In-fix expression.
- When Arithmetic Expression tree is traversed in post-order, We will obtain Post-fix expression.

Example: Expression Tree for (2 * (a − 1) + (3 * b))

Arithmetic Expression Trees (2/2)



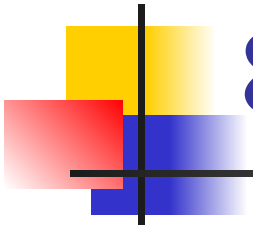
In-Order Traversal of Expression Tree Results In-fix Expression.

2 * (a – 1) + (3 * b)

Post-Order Traversal of Expression Tree Results Post-fix Expression.

2 a 1 – * 3 b * +

To convert In-fix Expression to Post-fix expression, First construct an expression tree using infix expression and then do tree traversal in post order.



8.13 Extended Binary Trees

- **Path length**
  - Path length of a node in a tree is defined as the number of edges that has to be traversed from the node to the root node.
- **External node**
  - The node with zero children is called an external node (or in other words, an external node is nothing but a leaf node).
- **Internal node**
  - A node with one or more children is called an internal node.

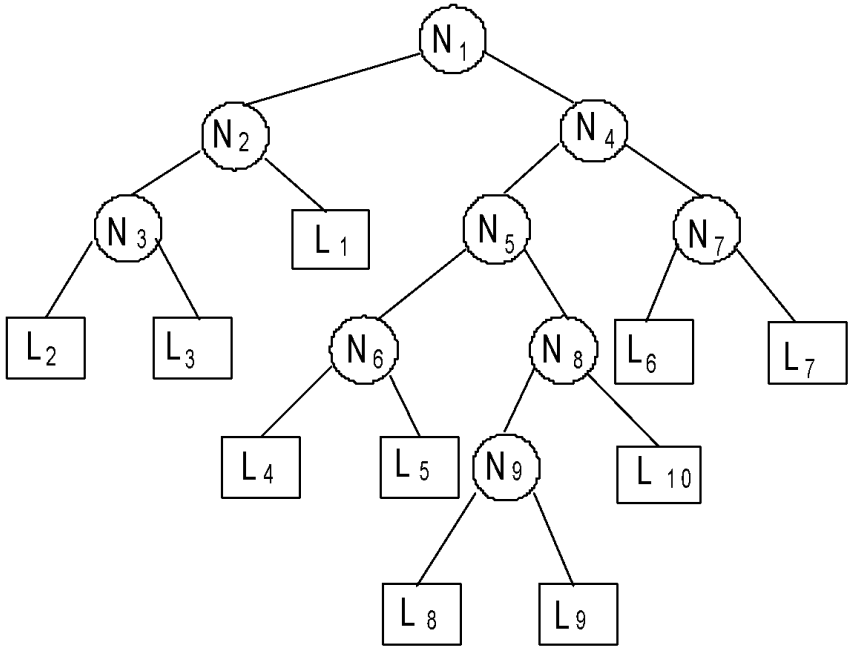
Extended Binary Trees

External node

- The node with zero children is called an external node (or in other words, an external node is nothing but a leaf node).

Internal node

- A node with one or more children is called an internal node.

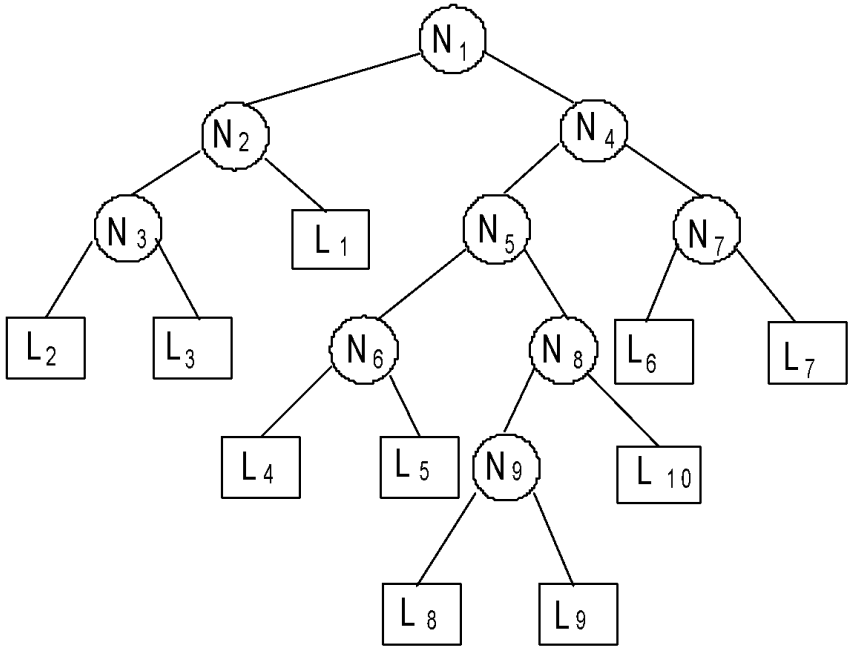


Extended Binary Trees

Path length

- Path length of a node in a tree is defined as the number of edges that has to be traversed from the node to the root node

Path length of $N_1 = 0$
Path length of $N_9 = 4$
Path length of $L_5 = 4$
Path length of $L_8 = 5$

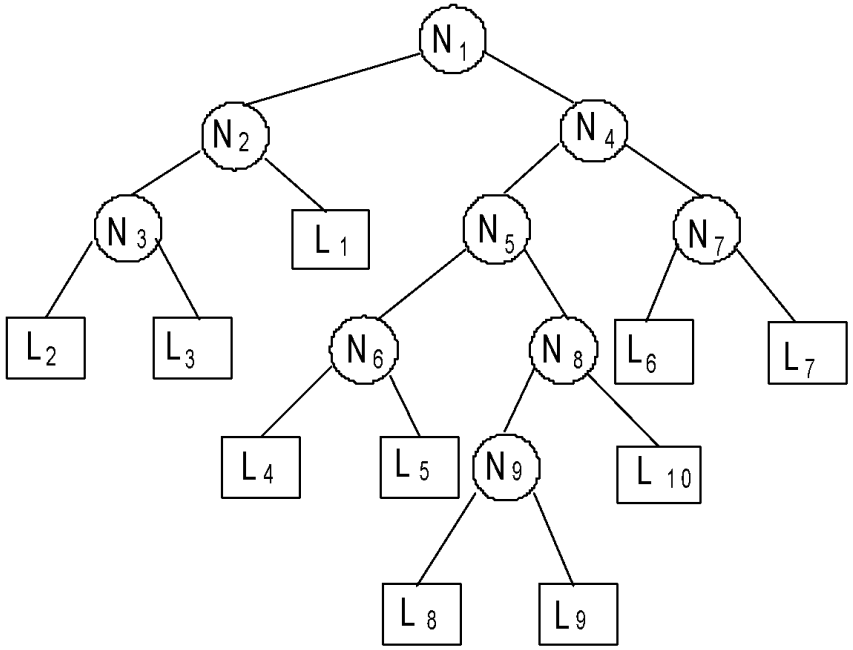


Extended Binary Trees

External path length

- External path length (let it be denoted as E) of a binary tree is defined as the sum of all path lengths, summed over each path from the root node of the tree to an external node

$$E = \sum_{i=1}^{N_E} P_i$$



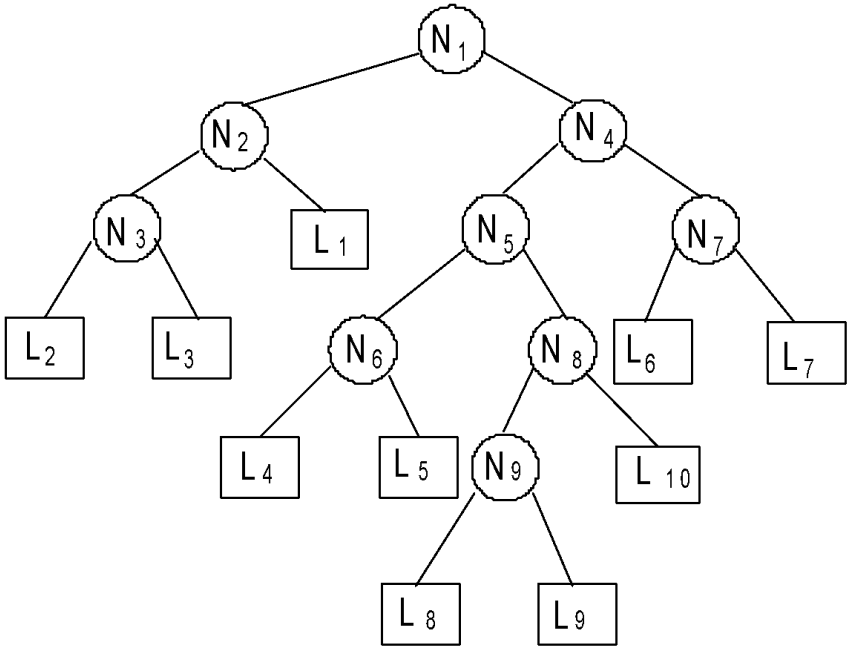
P_i denotes the path length of the i-th external node

$$E = \underset{L_1}{2} + \underset{L_2}{3} + \underset{L_3}{3} + \underset{L_4}{4} + \underset{L_5}{4} + \underset{L_6}{3} + \underset{L_7}{3} + \underset{L_8}{5} + \underset{L_9}{5} + \underset{L_{10}}{4} = 36$$

Extended Binary Trees

Internal path length

- Internal path length (let it be denoted as *I*) of a binary tree can be defined analogously as the sum of path lengths of all internal nodes in the tree



$$I = \sum_{i=1}^{N_I} P_i$$

P_i denotes the path length of the i-th internal node

N_1 N_2 N_3 N_4 N_5 N_6 N_7 N_8 N_9

$I = 0 + 1 + 2 + 1 + 2 + 3 + 2 + 3 + 4 = 18$

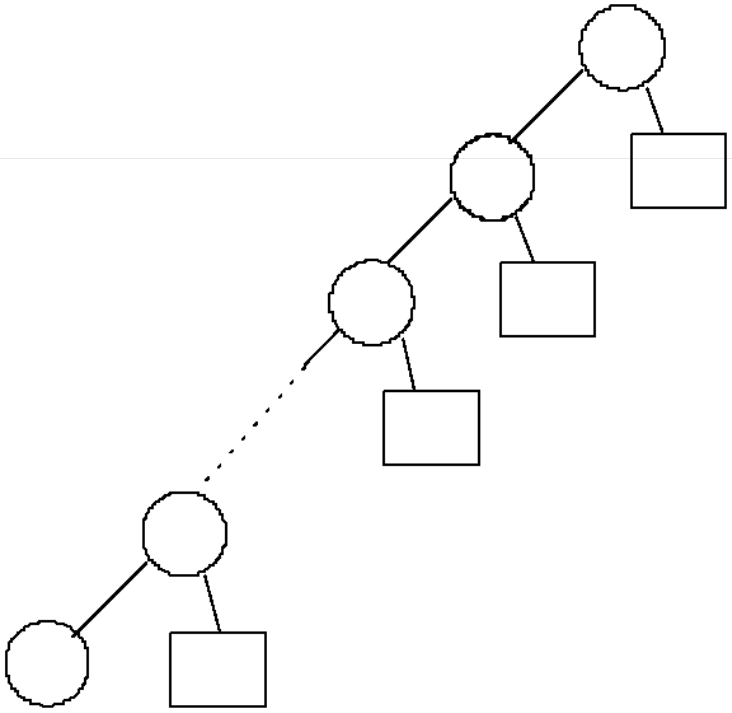
Extended Binary Trees

- **Properties**
  - In a binary tree with n internal nodes, if I denotes the internal path length, then external path length, $E = I + 2n$.

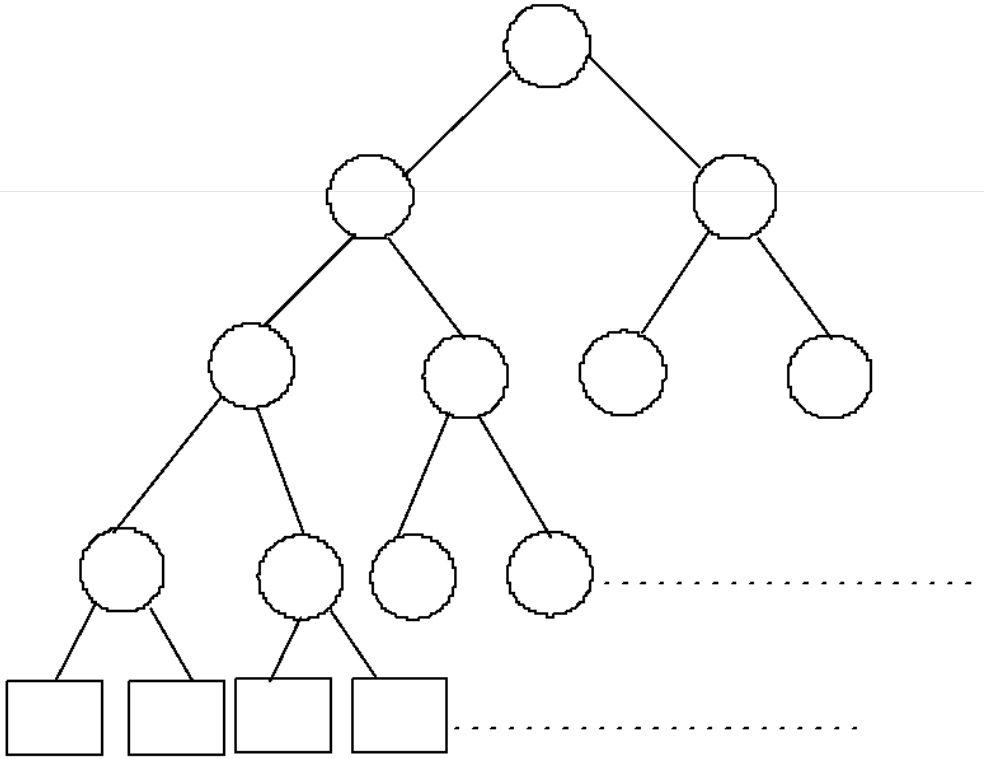
$$I_{\max} = 0 + 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

$$I_{\min} = 0 \cdot 1 + 2 \cdot 1 + 4 \cdot 2 + 8 \cdot 3 + \cdots = \sum_{i=0}^{l_{\max}} i 2^i$$

Extended Binary Trees



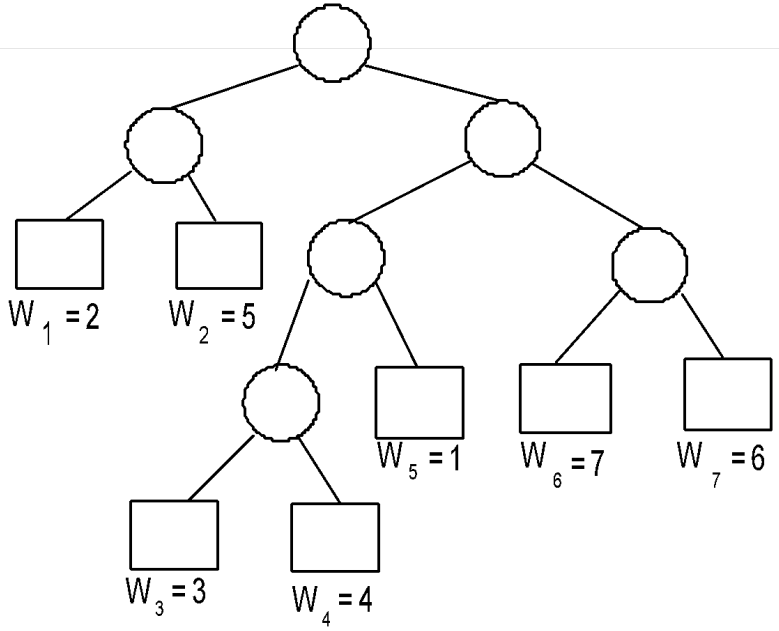
Binary tree with I_{max}



Binary tree with I_{min}

Weighted Binary Trees

- A binary tree is called weighted binary tree if a weight is assigned to each external node.



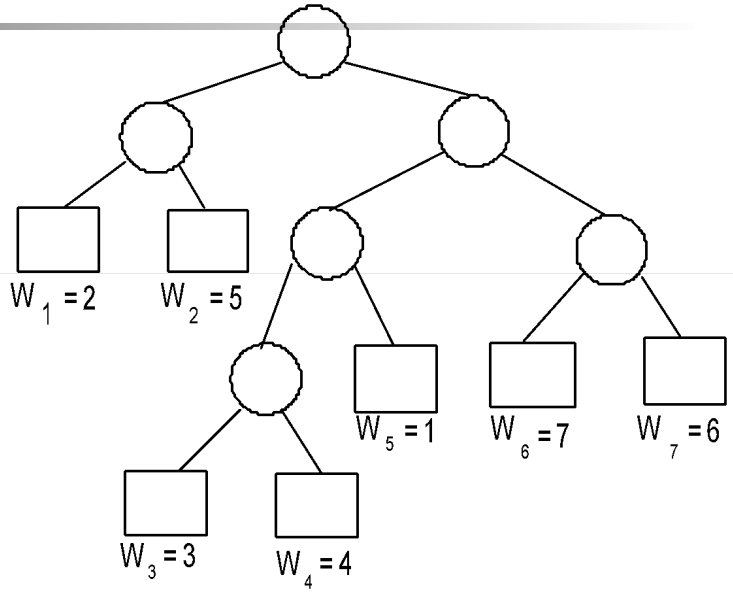
Weighted Binary Trees

Weighted path length

- The external path length with the weights are called *external weighted path length* (or, simply *weighted path length*).

$$P = \sum_{i=1}^{n} W_i L_i$$

where W_i is a weight of an external node n_i and its path length is L_i , n is the total number of external nodes



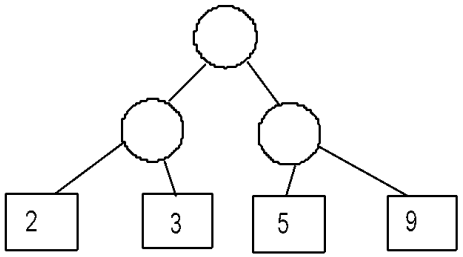
$P = W_1L_1 + W_2L_2 + W_3L_3 + W_4L_4 + W_5L_5 + W_6L_6 + W_7L_7$

$= 2\times2 + 5\times2 + 3\times4 + 4\times4 + 1\times3 + 7\times3 + 6\times3$

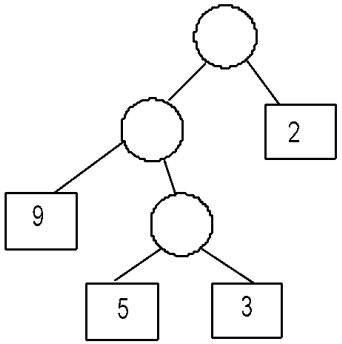
$= 84$

Minimum Weighted Binary Trees

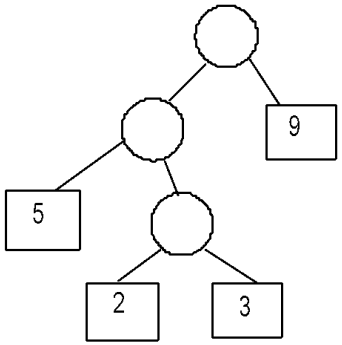
- The objective is to construct a weighted binary tree for a given set of weights of the external nodes, so that the weighted path length is minimum



(a) T_1

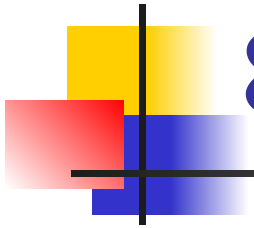


(b) T_2



(c) T_3

$P(T_1) = 38$ $P(T_2) = 44$ $P(T_3) = 34$



8.14 Huffman Tree

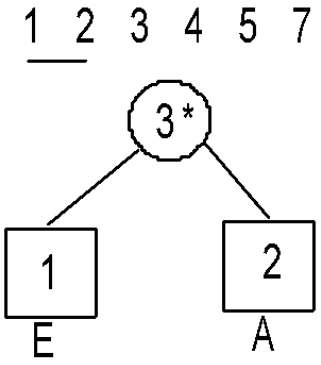
- A weighted binary tree according to Huffman's algorithm
  - Minimum weighted binary tree

- **Huffman's algorithm**
- Suppose, we have to construct a minimum weighted binary tree (Huffman tree) with n weights $W_1, W_2, \ldots, W_n$.
  - Sort the weights in ascending order.
  - Obtain a sub-tree with two minimum weights as the weights of external nodes.
  - Include the weighted path length of the sub-tree so obtained into the list of weights.
  - Repeat the procedure until the list contains single weight.

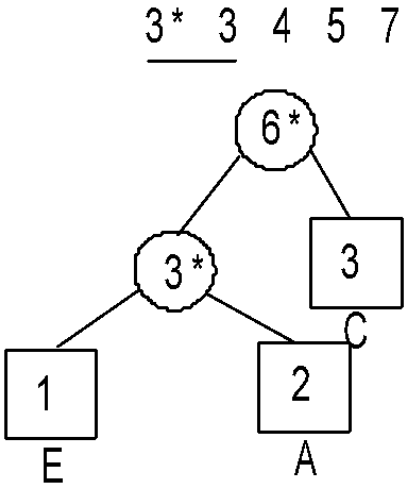
Huffman's Algorithm: Example

- **Input**

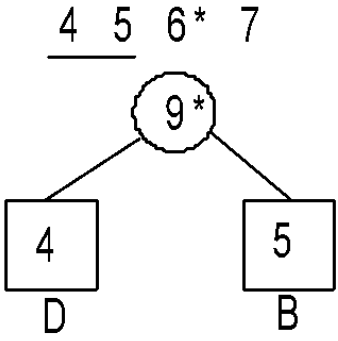
External node:	*A*	*B*	*C*	*D*	*E*	*F*
Weight:	2	5	3	4	1	7



(a)

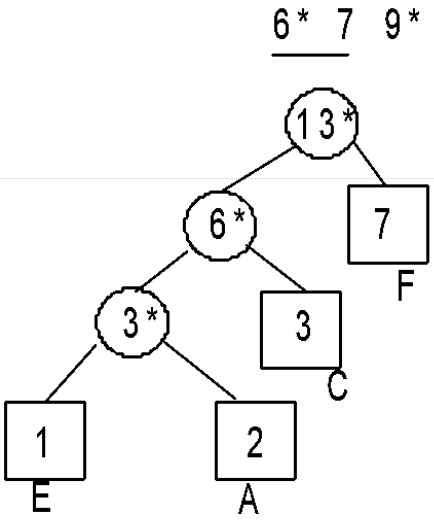


(b)

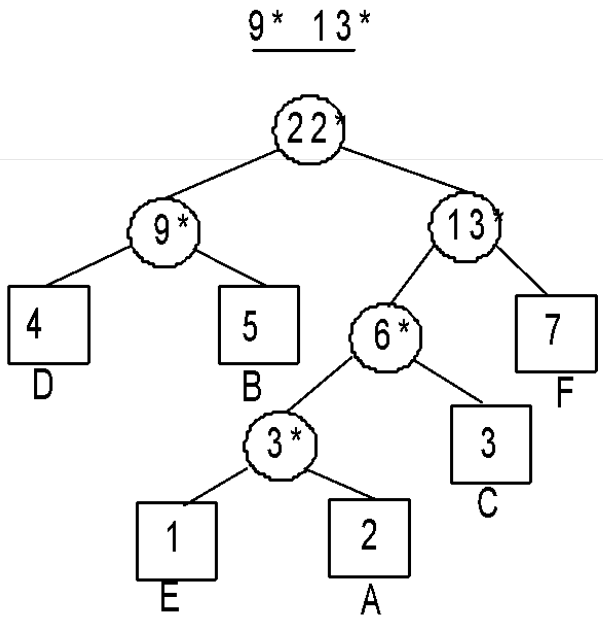


(c)

Huffman's Algorithm: Example



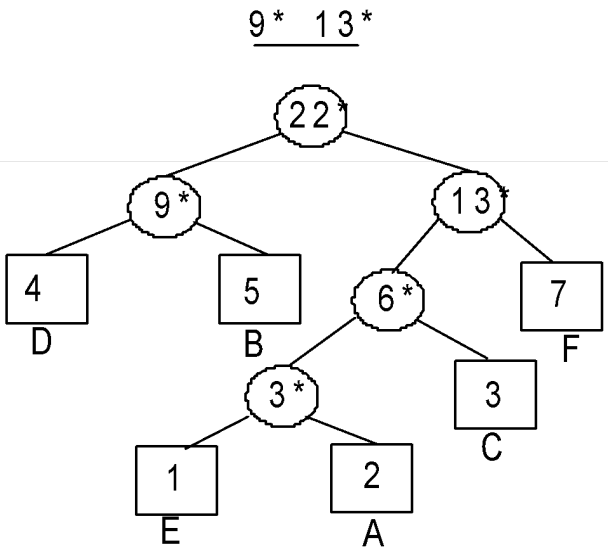
(d)



(e)

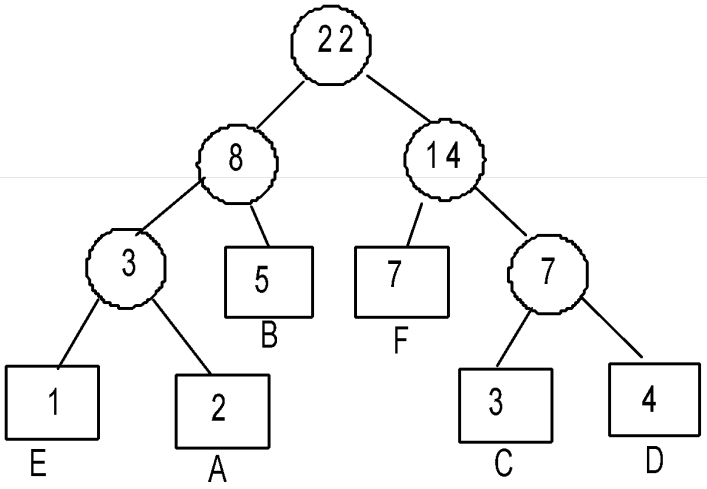
$P = 22 + 9 + 13 + 6 + 3 = 53$

Huffman's Algorithm: Example



Huffman tree according to Huffman's algorithm

$P = 22 + 9 + 13 + 6 + 3 = 53$

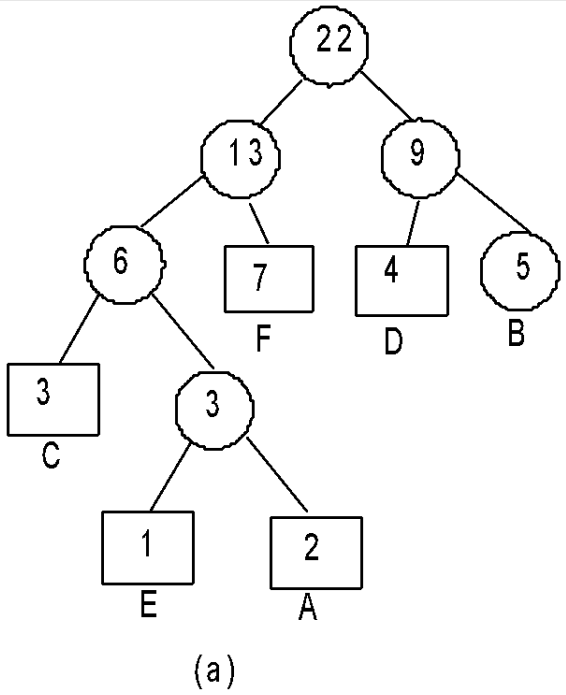


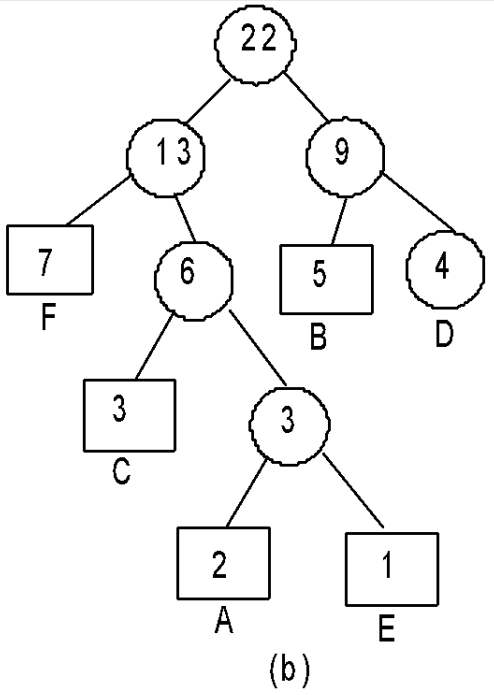
A weighted binary tree but not Huffman tree

$P = 22 + 8 + 14 + 3 + 7 = 54$

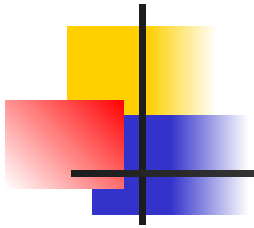
Huffman's Algorithm: Example

- Huffman tree is not necessarily unique for a given set weights





Application of Huffman Tree: Huffman Coding



- To obtain an optimal set of codes for symbols S_1, S_2, ..., S_n which constitute messages. Each code is a binary string (combinations of 0's and 1's) which will be used for transmission of messages
  - Fixed length coding
  - Variable length coding

Application of Huffman Tree: Huffman Coding

- Fixed length coding

*S*1	0000	
*S*2	0001	
*S*3	0010	
*S*4	0011	
*S*5	0100	
S6		0101
*S*7	0110	
*S*8	0111	
*S*9	1000	
*S*10	1001	
*S*11	1010	

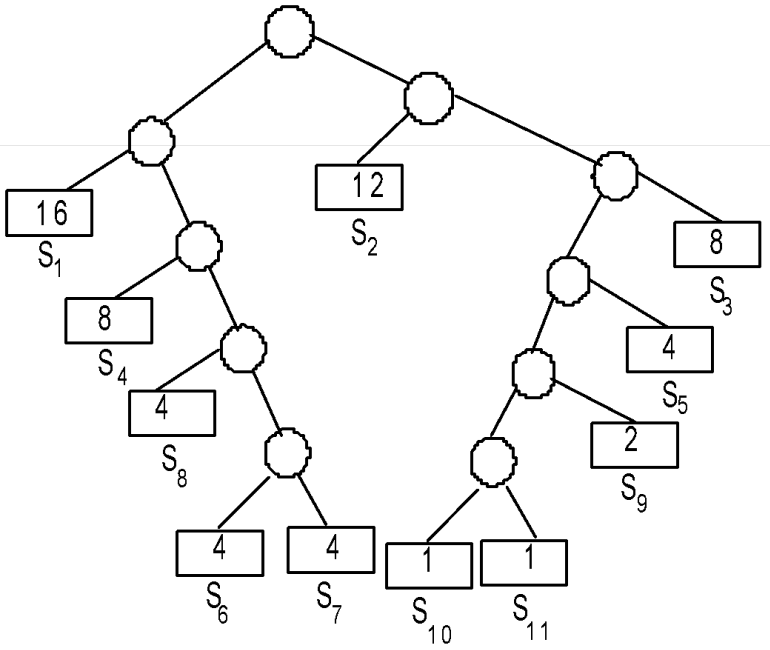
Application of Huffman Tree: Huffman Coding

- Variable length coding

*S*1	1/4	00
*S*2	3/16	10
*S*3	1/8	111
*S*4	1/8	010
*S*5	1/16	1101
*S*6	1/16	01110
*S*7	1/16	01111
*S*8	1/16	0110
*S*9	1/32	11001
*S*10	1/64	110000
*S*11	1/64	110001

Application of Huffman Tree: Huffman Coding

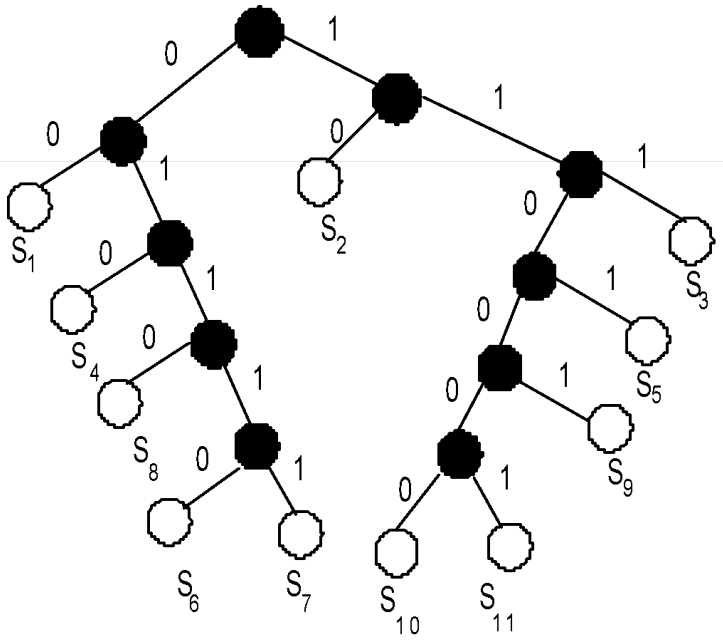
*S*1	16
*S*2	12
*S*3	8
*S*4	8
*S*5	4
*S*6	4
*S*7	4
*S*8	4
*S*9	2
*S*10	1
*S*11	1



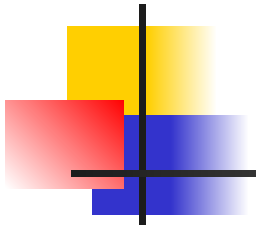
Huffman tree

Application of Huffman Tree: Huffman Coding

*S*1	16	00
*S*2	12	10
*S*3	8	111
*S*4	8	010
*S*5	4	1101
*S*6	4	01110
*S*7	4	01111
*S*8	4	0110
*S*9	2	11001
*S*10	1	110000
*S*11	1	110001



Huffman coding

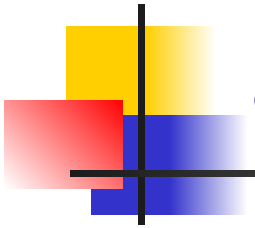


Huffman Coding

Advantages

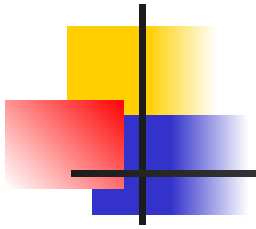
- The Huffman coding has the minimum average length

- It does not require any end-of-character delimiter although the binary codes are of variable length codes. This is because of its unique prefix property



8.15 Threaded Binary Trees (1/2)

- Two many null pointers in current representation of binary trees
  - n: number of nodes
  - number of non-null links: n-1
  - total links: 2n
  - null links: 2n-(n-1)=n+1
- Replace these null pointers with some useful “threads”.

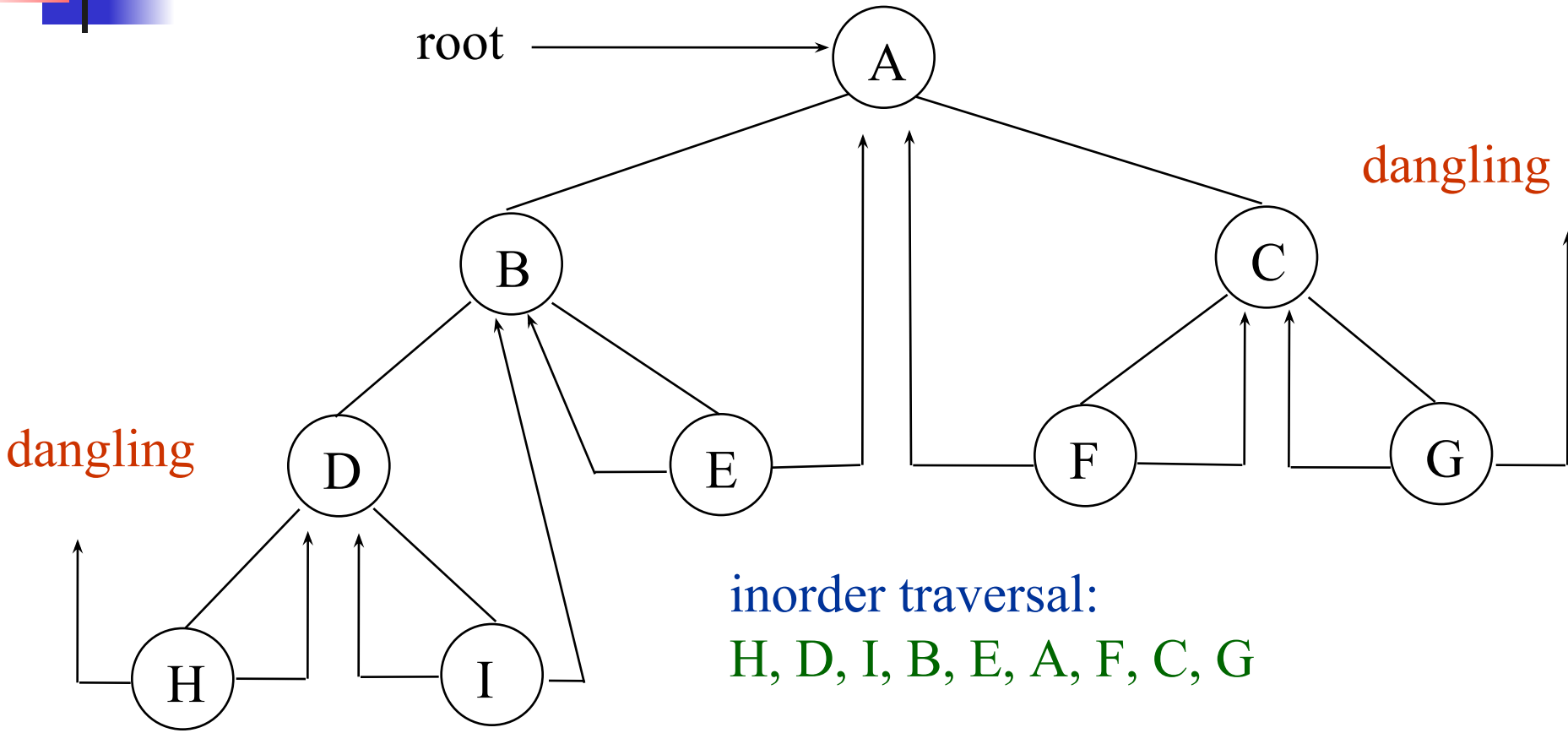


Threaded Binary Trees (2/2)

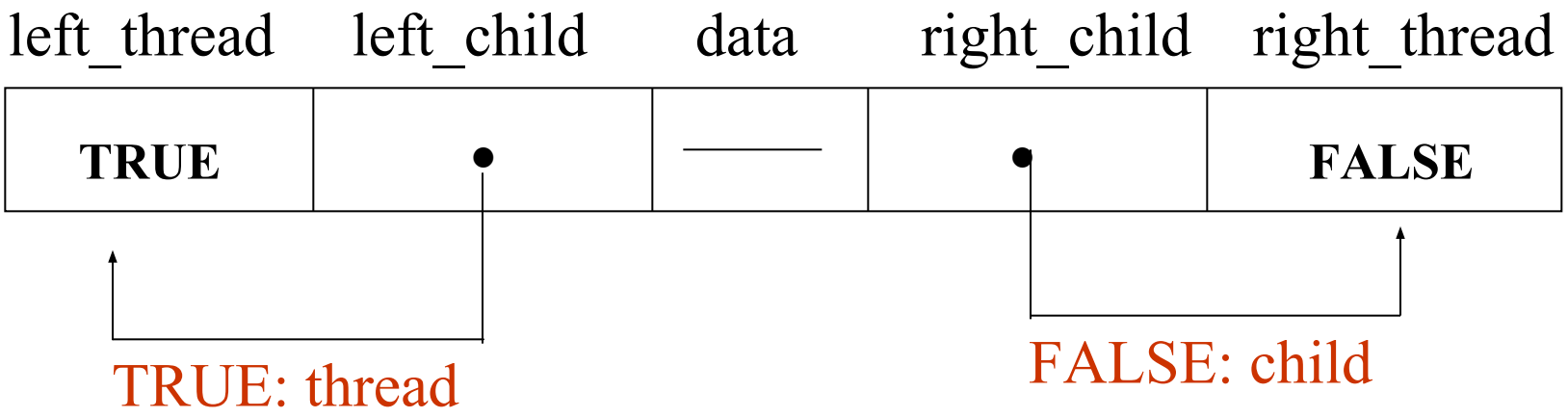
If `ptr->left_child` is null,
replace it with a pointer to the node that would be visited *before* `ptr` in an *inorder traversal*

If `ptr->right_child` is null,
replace it with a pointer to the node that would be visited *after* `ptr` in an *inorder traversal*

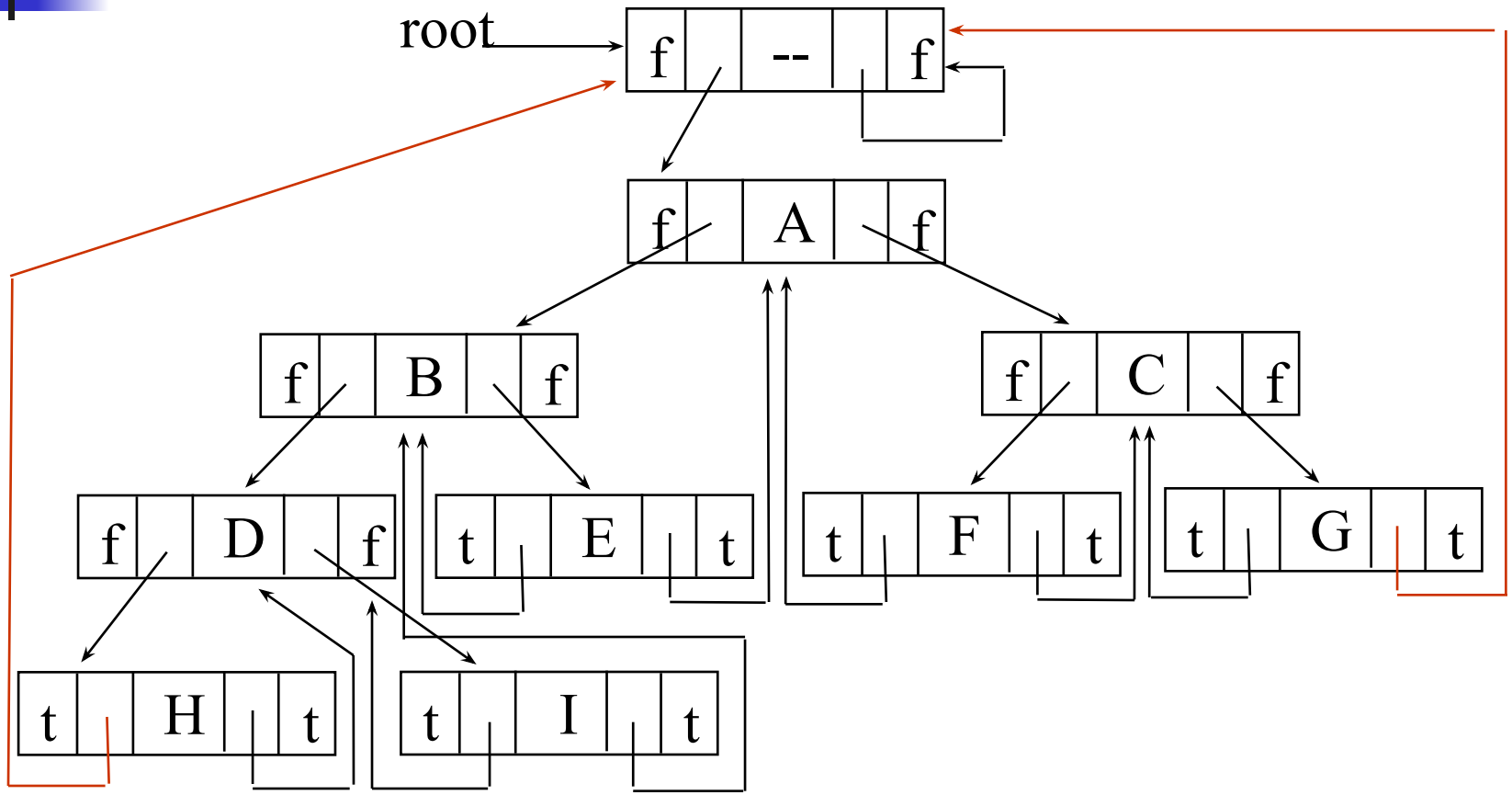
A Threaded Binary Tree



Data Structures for Threaded BT

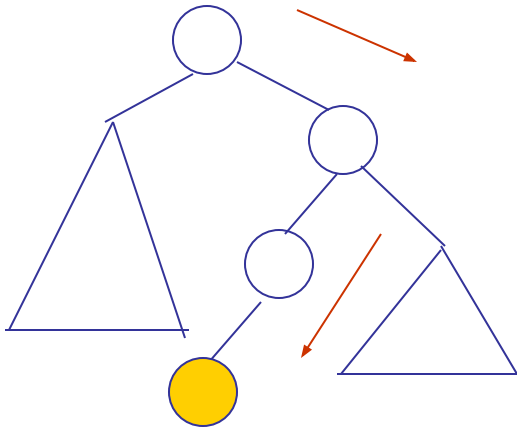


Memory Representation of A Threaded BT



Next Node in Threaded BT

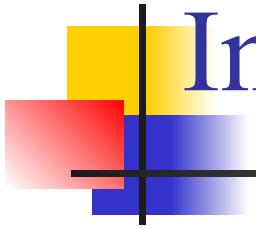
```
threaded_pointer insucc(threaded_pointer tree)
{
  threaded_pointer temp;
  temp = tree->right_child;
  if (!tree->right_thread)
    while (!temp->left_thread)
      temp = temp->left_child;
  return temp;
}
```



Inorder Traversal of Threaded BT

```
void tinorder(threaded_pointer tree)
{
/* traverse the threaded binary tree inorder *
     threaded_pointer temp = tree;
     for (;;) {
          temp = insucc(temp);
          if (temp==tree) break;
          printf("%3c", temp->data);
     }
}
```

O(n)

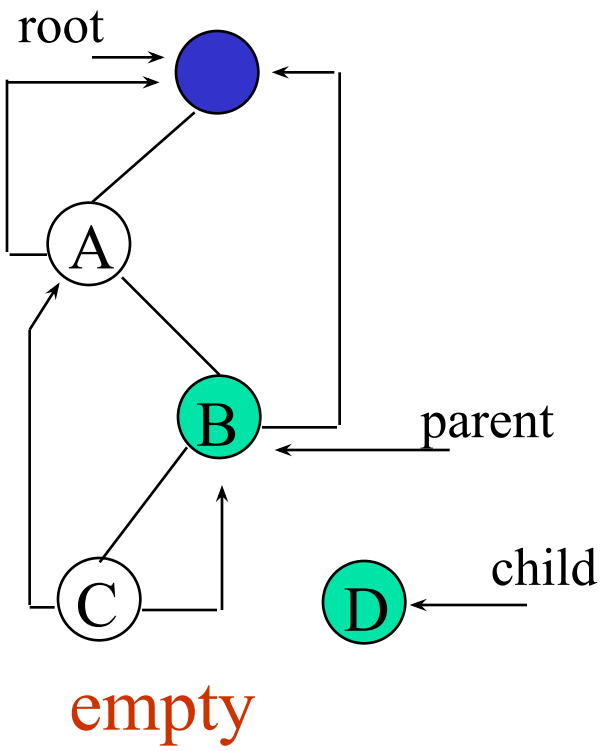


Inserting Nodes into Threaded BTs

- Insert `child` as the right child of node `parent`
  - change `parent->right_thread` to FALSE
  - set `child->left_thread` and `child->right_thread` to TRUE
  - set `child->left_child` to point to `parent`
  - set `child->right_child` to `parent->right_child`
  - change `parent->right_child` to point to `child`

Examples

Insert a node D as a right child of B.



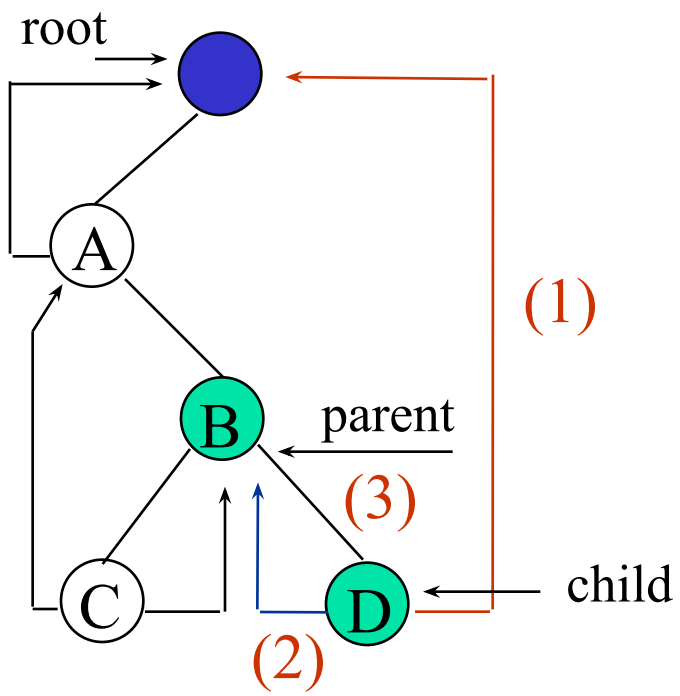
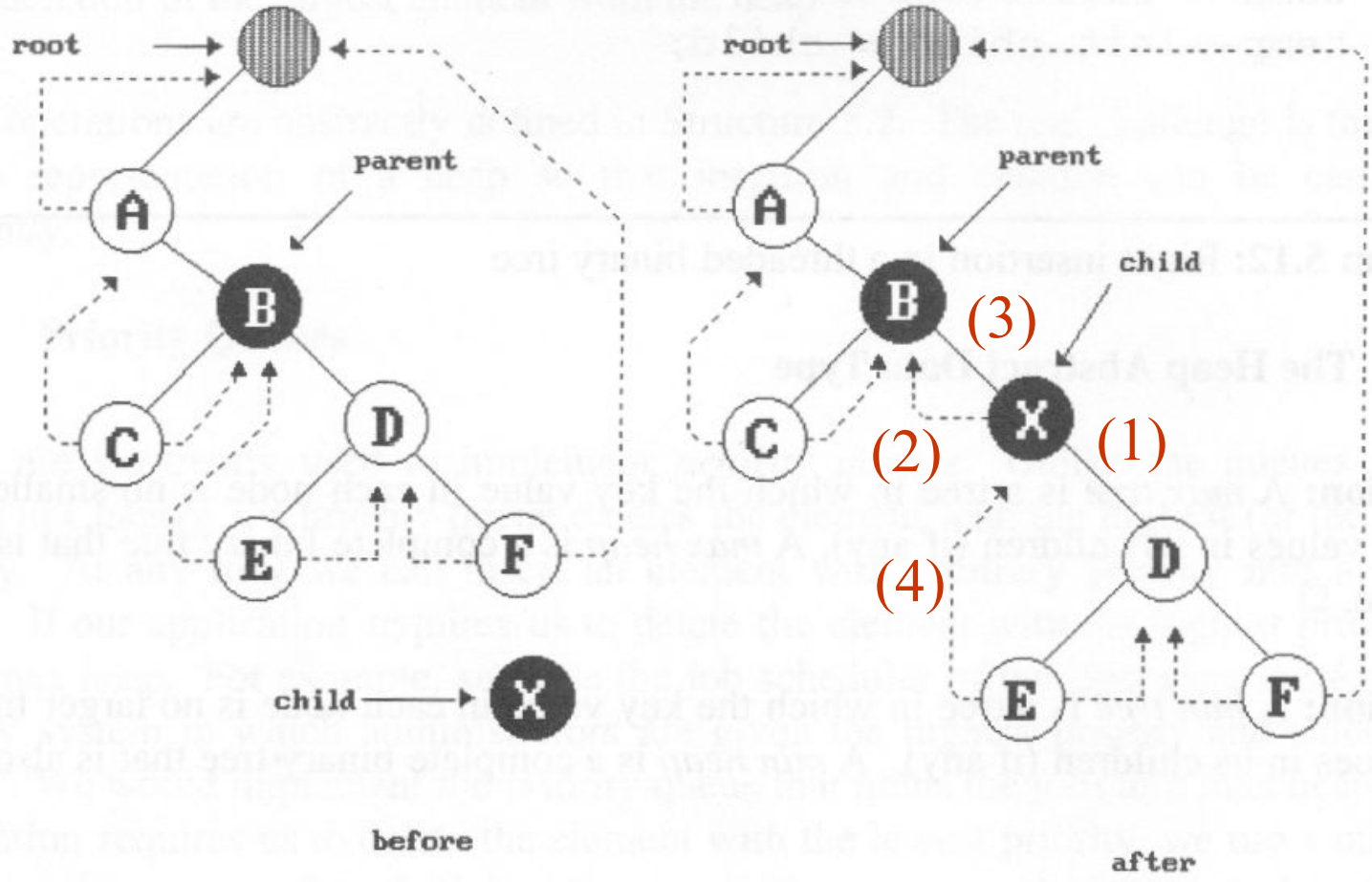


***Figure 5.24:** Insertion of child as a right child of parent in a threaded binary tree (p.217)



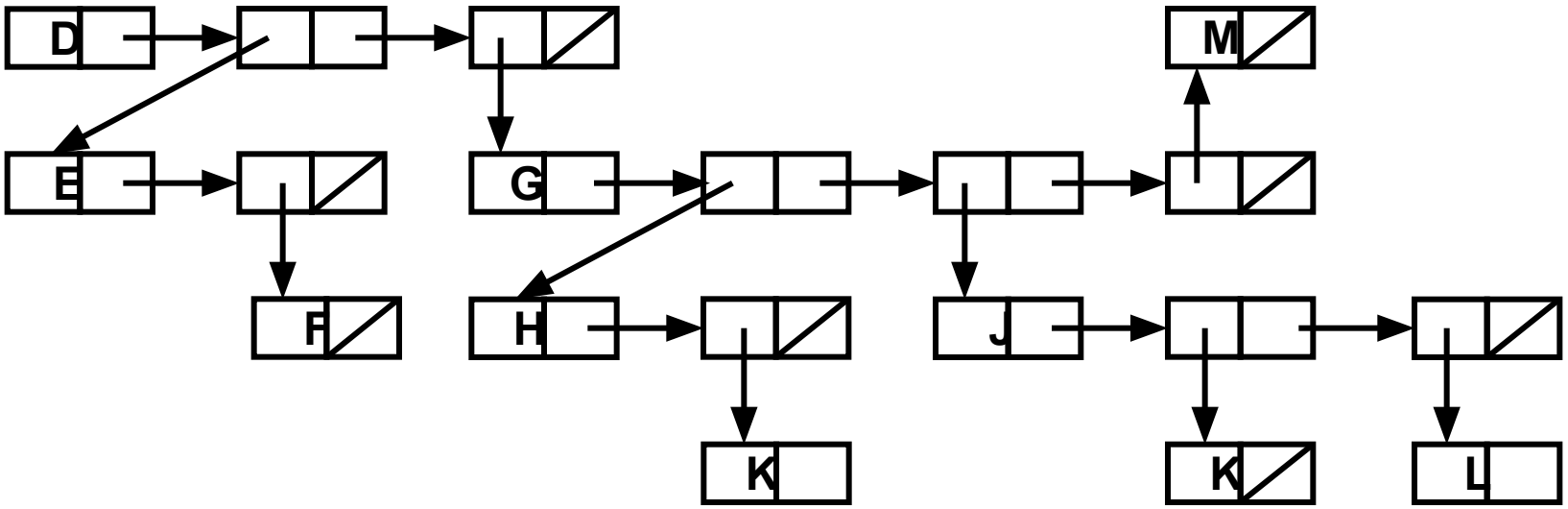
nonempty (b)

8.19 General Trees

- A general tree is an ordered tree whose set of nodes cannot be empty.
- In a general tree, there is no limit to the number of children that a node can have.
- Representing a general tree by linked lists:
  - Each node has a linked list of the subtrees of that node.

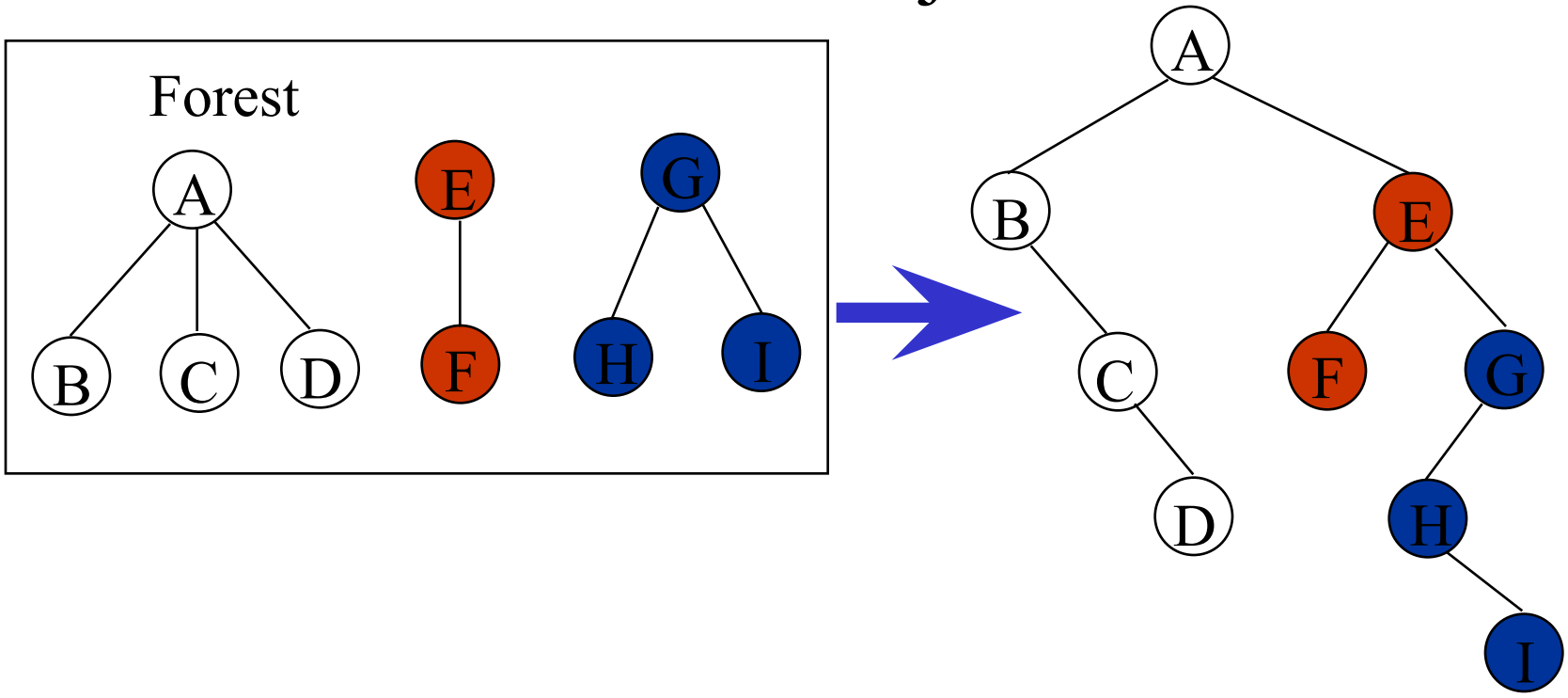
General Trees (Contd.)

- An example of a general tree:

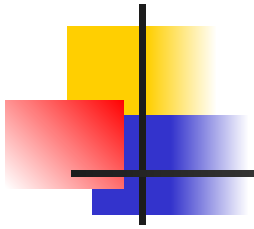


Forest

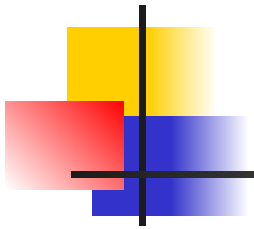
- A forest is a set of n >= 0 disjoint trees



Transform a forest into a binary tree

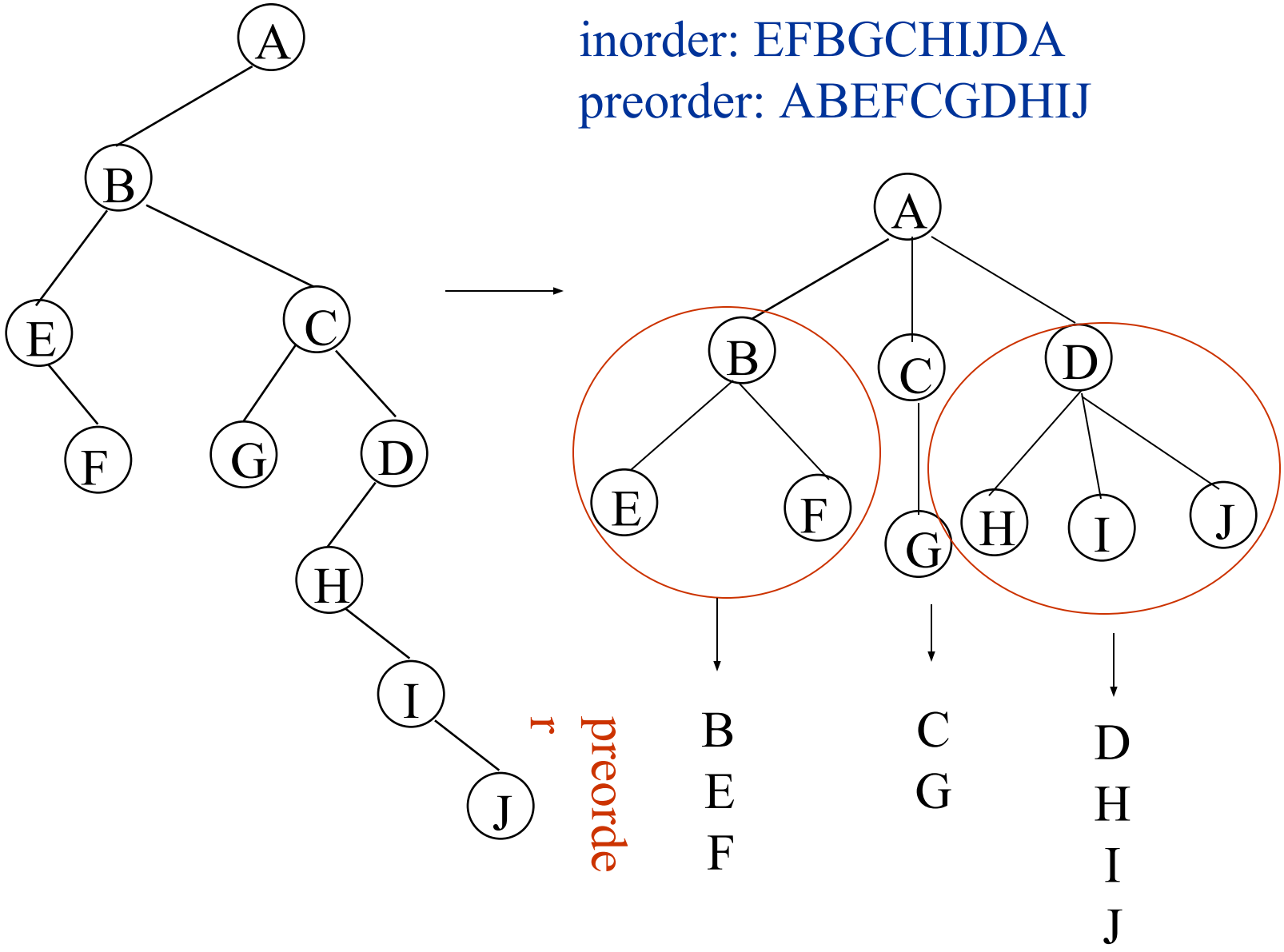


- T1, T2, …, T*n*: a forest of trees
  B(T1, T2, …, Tn): a binary tree corresponding to this forest
- algorithm
  (1) empty, if n = 0
  (2) has root equal to root(T1)
  has left subtree equal to B(T11,T12,…,T1*m*)
  has right subtree equal to B(T2,T3,…,Tn)



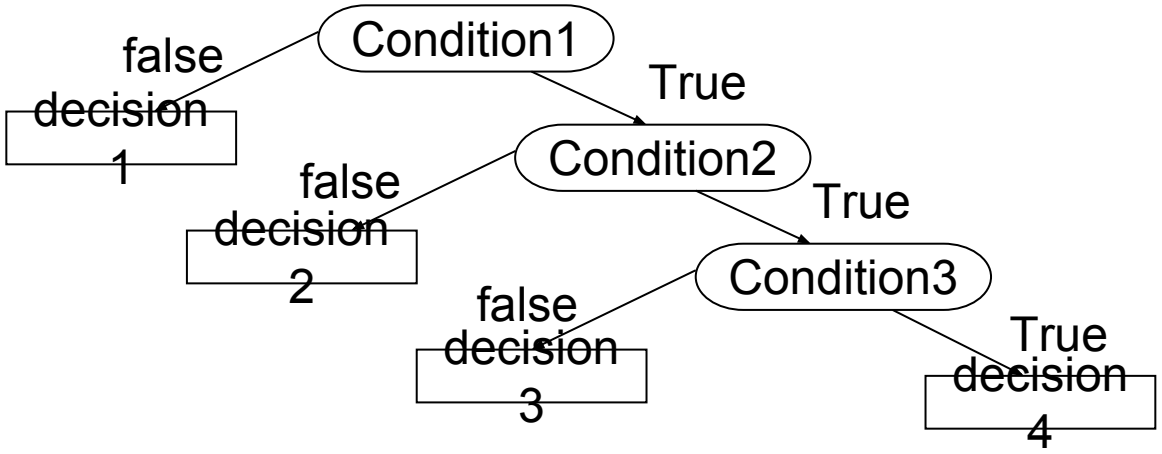
Forest Traversals

- Preorder
  - If F is empty, then return
  - Visit the root of the first tree of F
  - Taverse the subtrees of the first tree in tree preorder
  - Traverse the remaining trees of F in preorder
- Inorder
  - If F is empty, then return
  - Traverse the subtrees of the first tree in tree inorder
  - Visit the root of the first tree
  - Traverse the remaining trees of F is indorer


A
B
E
C
F
G
D
H
I
J
inorder: EFBGCHIJDA
preorder: ABEFCGDHIJ
A
B
C
D
E
F
G
H
I
J
r
preorde
B
E
F
C
G
D
H
I
J

Application of Binary Trees

- Binary trees have many important uses. Two examples are:
  1. Binary decision trees.
     - Internal nodes are conditions. Leaf nodes denote decisions.



- Expression Tree

